



ESTACIÓN METEOROLÓGICA

FUNDAMENTOS DE IoT



ANTONIO GUZMÁN RIVERA

19339839

901

DOCENTE:

Luis Gerardo Navarrete Bautista

ÍNDICE

INTRODUCCIÓN	7
1. Primera Capa: Percepción.....	9
1.1 ¿Qué es la capa de percepción?	9
1.2 Componentes de la capa de percepción	9
1.3 Sensor AHT20 — Temperatura y Humedad Relativa.....	10
1.3.1 ¿Qué chip es y qué integra?	10
1.3.2 ¿Cómo mide la temperatura?	11
1.3.3 ¿Cómo mide la humedad relativa?.....	11
1.3.4 ¿Cómo se comunica con el ESP32?.....	12
1.4 Sensor BMP280 — Presión Atmosférica y Temperatura.....	12
1.4.1 ¿Qué chip es y qué integra?	12
1.4.2 ¿Cómo mide la presión?.....	13
1.4.3 Problema resuelto: bus I2C aislado.....	13
1.5 Encoder Magnético AS5600 — Dirección del Viento	14
1.5.1 ¿Qué chip es y qué integra?	14
1.5.2 ¿Cómo mide la dirección del viento?.....	15
1.6 Módulo KY-025 con Reed Switch — Velocidad del Viento (Anemómetro).....	16
1.6.1 ¿Qué es el módulo KY-025 y qué integra?	16
1.6.2 ¿Cómo mide la velocidad del viento?	18
1.7 Módulo KY-025 con Reed Switch — Precipitación Pluvial (Pluviómetro)	19
1.7.1 ¿Qué es el pluviómetro de balancín y cómo funciona?	19
1.7.2 Cálculo de la precipitación acumulada	19
1.8 Pantalla OLED SSD1306 — Visualización Local.....	20
1.8.1 ¿Qué es el SSD1306 y qué integra?.....	20
1.8.2 Funcionamiento del display en la estación	21
1.9 Decisión de diseño: dos buses I2C independientes.....	21
2. SEGUNDA CAPA: RED	23
2.1 ¿Qué hace esta capa?	23
2.2 Tecnología de red principal — WiFi 802.11n.....	23
2.2.1 ¿Qué chip maneja el WiFi en el ESP32?	23

2.2.2 Especificaciones del módulo WiFi integrado	24
2.2.3 ¿Cómo establece la conexión el ESP32?	24
2.3 Protocolo de transporte — HTTPS sobre TCP/IP	25
2.3.1 Modelo de capas de la comunicación ESP32 → Firebase	25
2.3.2 ¿Por qué HTTPS y no HTTP?	25
2.4 Estructura del flujo de datos hacia Firebase	25
2.4.1 Serialización en formato JSON.....	25
2.4.2 Estrategia de envío — dos canales con distinta frecuencia	26
2.5 Recepción de datos en el celular — Firebase SDK	27
2.5.1 ¿Cómo llegan los datos al teléfono?	27
2.5.2 Protocolo de comunicación Firebase → celular	27
2.6 Configuración WiFi por BLE — protocolo del celular al ESP32.....	27
2.6.1 ¿Para qué sirve BLE en esta capa?	27
2.6.2 ¿Qué pin usa el BLE en el ESP32?	28
2.6.3 Especificaciones del módulo BLE integrado	28
2.6.4 ¿Cómo funciona la comunicación BLE entre el celular y el ESP32?	28
2.7 Tolerancia a fallos en la capa de red	29
3. TERCERA CAPA: PROCESAMIENTO.....	30
3.1 ¿Qué hace esta capa?	30
3.2 El microcontrolador ESP32 — hardware de procesamiento	30
3.2.1 ¿Qué es el ESP32 y qué integra?	30
3.2.2 Mapa de pines — conexiones físicas del ESP32	31
3.3 Declaración de variables globales.....	32
3.3.1 Variables de conteo de pulsos — declaradas como volatile.....	32
3.3.2 Variables de resultado meteorológico	32
3.3.3 Variables de control de tiempo.....	33
3.3.4 Variables de estado del Sistema	34
3.4 Rutinas de interrupción (ISR)	34
3.5 Operaciones matemáticas de procesamiento	35
3.5.1 Conversión de pulsos a velocidad del viento	35
3.5.2 Conversión de pulsos a precipitación en milímetros	36

3.5.3 Conversión del valor crudo del AS5600 a grados y punto cardinal.....	36
3.5.4 Conversión de la presión cruda del BMP280 a hPa.....	37
3.6 Validación de datos y recuperación automática	37
3.6.1 Función de validación	37
3.6.2 Recuperación automática del bus I2C.....	38
3.7 Estructura del programa — setup() y loop()	38
3.7.1 setup() — inicialización única al arranque	38
3.7.2 loop() — ejecución continua y concurrente	39
3.8 Generación de timestamps ISO 8601.....	40
4. CUARTA CAPA: APLICACIÓN	41
4.1 ¿Qué hace esta capa?	41
4.2 Backend — Firebase Realtime Database	41
4.3. Aplicación web.....	42
4.3.1 Inicialización y conexión con Firebase.....	42
4.3.2 Tarjetas de sensores y transición numérica animada	43
4.3.3 Fondo dinámico animado con Canvas	43
4.3.4 Rosa de los vientos SVG animada	44
4.3.5 Sistema de alertas y frases contextuales	45
4.3.6 Historial — gráficas y tabla de registros.....	46
4.4 Aplicación móvil — Flutter	46
4.4.1 Arquitectura de la aplicación	46
4.4.2 Modelo de datos — WeatherData	47
4.4.3 Pantalla principal — HomeScreen.....	49
4.4.4 Widget reutilizable — SensorCard	50
4.4.5 Sistema de alertas y frases contextuales en Flutter.....	51
4.4.6 Navegación entre pantallas	52
4.5 Flujo completo de datos en la capa de aplicación.....	52
5. Código fuente – Repositorio del proyecto en GitHub	53
Referencias.....	54

Índice de figuras

Ilustración 1 Módulo AHT20 / módulo dual AHT20+BMP280.....	10
Ilustración 2 Stevenson Screen para el módulo BMP280.....	11
Ilustración 3 Módulo AS5600 con imán de neodimio	14
Ilustración 4 Velea con imán para conocer dirección del viento	15
Ilustración 5 Módulo KY-025	16
Ilustración 6 Anemómetro de cazoletas	17
Ilustración 7 Pluviómetro.....	19
Ilustración 8 Pantalla OLED SSD1306.....	20
Ilustración 9 Estructura del archivo JSON que se envía a Firebase	26
Ilustración 10 Placa de desarrollo ESP32	30
Ilustración 11 Protección del valor durante la lectura del contador.....	35
Ilustración 12 Dashboard meteorológico	42
Ilustración 13 Inicialización de Firebase	42
Ilustración 14 Tarjetas individuales para mostrar datos.....	43
Ilustración 15 Función setSkyState.....	44
Ilustración 16 Rosa de los vientos animada.....	45
Ilustración 17 Función cargarHistorial() para los datos históricos	46
Ilustración 18 Modelo de datos WeatherData	48
Ilustración 19 HomeScreen de la app Móvil.....	49
Ilustración 20 StatelessWidget para el HomeScreen	50
Ilustración 21 Widget reutilizable — SensorCard	50
Ilustración 22 Container para cada tarjeta	51

Índice de tablas

Tabla 1 Componentes de la capa de percepción	9
Tabla 2 Especificaciones del módulo AHT20.....	11
Tabla 3 Especificaciones del módulo BMP280	13
Tabla 4 Especificaciones de módulo AS5600	15
Tabla 5 Especificaciones del KY-025 para anemómetro	17
Tabla 6 Especificaciones de la pantalla OLED SSD1306	21
Tabla 7 Buses I2C independientes	22
Tabla 8 Especificaciones del WIFI integrado	24
Tabla 9 Modelo de capas y su protocolo utilizado.....	25
Tabla 10 Estrategia de envío en dos canales con distinta frecuencia.....	26
Tabla 11 Protocolo de comunicación de Firebase a Movil	27
Tabla 12 Especificaciones del módulo BLE	28
Tabla 13 Características del servicio BLE implementadas	28
Tabla 14 Mecanismos de recuperación automática	29
Tabla 15 Especificaciones del ESP32	31
Tabla 16 Conexiones físicas del ESP32.....	31
Tabla 17 Variables globales de conteo	32
Tabla 18 Variables de resultado	32
Tabla 19 Variables de control de tiempo	33
Tabla 20 Variables de estado del sistema	34
Tabla 21 Rutinas de servicio de interrupción.....	34
Tabla 22 Rango de ángulo para determinar punto cardinal	36
Tabla 23 Validaciones de datos	37
Tabla 24 Pasos para recuperar el bus I2C automáticamente	38
Tabla 25 Estructura del programa: setup().....	38
Tabla 26 Estructura del programa: loop()	39
Tabla 27 Estados para fondo diáamico con Canvas.....	43
Tabla 28 Sistema de alertas y frases contextuales.....	45
Tabla 29 Arquitectura de carpetas de la aplicacion móvil	47
Tabla 30 Despliegue en distintas plataformas.....	52

INTRODUCCIÓN

En la actualidad, el Internet de las Cosas (IoT, por sus siglas en inglés) ha transformado la manera en que los sistemas tecnológicos interactúan con el entorno físico, permitiendo que dispositivos electrónicos capten, procesen y transmitan información del mundo real hacia plataformas digitales de forma autónoma y en tiempo real. Según Ashton (2009), quien acuñó el término Internet of Things, la verdadera revolución de esta tecnología radica en dotar a los objetos cotidianos de la capacidad de percibir y comunicar información sin intervención humana directa. Esta convergencia entre el mundo físico y el digital ha abierto posibilidades enormes en áreas como la agricultura, la salud, la domótica y, de manera particular, la meteorología.

El monitoreo climático ha sido históricamente una actividad reservada a instituciones gubernamentales y centros de investigación con infraestructura costosa y especializada. Sin embargo, gracias al avance en la miniaturización de sensores, la reducción de costos en microcontroladores y la disponibilidad de plataformas en la nube de acceso gratuito, hoy es posible construir sistemas de monitoreo meteorológico funcionales, precisos y accesibles desde cualquier dispositivo conectado a internet. Al respecto, Patel et al. (2016) señalan que la proliferación de microcontroladores de bajo costo con conectividad inalámbrica integrada ha democratizado el desarrollo de sistemas embebidos para monitoreo ambiental, reduciendo significativamente las barreras de entrada tanto económicas como técnicas.

El presente documento describe el diseño, desarrollo e implementación de una estación meteorológica basada en IoT, desarrollada como proyecto universitario. El sistema utiliza un microcontrolador ESP32 como unidad central de procesamiento, al cual se conectan sensores especializados para medir las principales variables climáticas: temperatura y humedad relativa mediante el módulo AHT20, presión atmosférica mediante el BMP280, dirección del viento mediante el encoder magnético AS5600, velocidad del viento mediante un anemómetro con sensor reed switch KY-025, y precipitación pluvial mediante un pluviómetro de balancín con el mismo tipo de sensor. De acuerdo con Espressif Systems (2023), el ESP32 integra en un solo chip un procesador dual-core Xtensa LX6 a 240 MHz, módulo WiFi 802.11 b/g/n y Bluetooth 4.2, lo que lo convierte en una plataforma idónea para aplicaciones IoT que requieren procesamiento local y conectividad simultánea. Los datos capturados son procesados en el microcontrolador y transmitidos en tiempo real a Firebase Realtime Database, desde donde son consumidos por una aplicación web y una aplicación móvil desarrollada en Flutter.

La justificación de este proyecto radica en la necesidad de contar con información climática local, precisa y accesible en regiones donde las estaciones meteorológicas oficiales son escasas o están alejadas de las zonas de interés. En este sentido, Mendez et al. (2020) destacan que las redes de estaciones meteorológicas de bajo costo basadas en IoT representan una alternativa viable y complementaria a las redes institucionales tradicionales, especialmente en países en desarrollo donde la densidad de estaciones oficiales es insuficiente para cubrir la variabilidad climática regional. Chilpancingo, Guerrero, al ser una ciudad con variaciones climáticas marcadas a lo largo

del año, representa un caso de uso real donde una estación de este tipo puede aportar valor tanto a nivel personal como comunitario, proporcionando datos útiles para la toma de decisiones cotidianas relacionadas con el clima.

El objetivo general del proyecto es diseñar e implementar un sistema de monitoreo meteorológico basado en la arquitectura IoT, capaz de capturar variables físicas del entorno, procesarlas en un sistema embebido y visualizarlas en tiempo real a través de plataformas web y móvil, aplicando los conceptos de las capas de percepción, red y aplicación propios de los sistemas IoT. Este enfoque por capas es consistente con el modelo de referencia IoT propuesto por el Instituto de Ingenieros Eléctricos y Electrónicos (IEEE, 2015), que estructura los sistemas IoT en niveles funcionales desde la percepción física hasta la aplicación de usuario.

El documento está organizado en tres capas que describen la arquitectura completa del sistema. La Capa de Percepción abarca los sensores físicos y los mecanismos mediante los cuales capturan y transmiten las variables climáticas al microcontrolador. La Capa de Red o Transporte describe cómo el ESP32 establece la conexión WiFi y envía los datos de forma segura hacia Firebase a través del protocolo HTTPS. Finalmente, la Capa de Aplicación presenta las plataformas de visualización desarrolladas — la aplicación web con dashboard interactivo y la aplicación móvil — así como la estructura de la base de datos y las decisiones de diseño que garantizan el funcionamiento eficiente del sistema en su conjunto.

1. PRIMERA CAPA: PERCEPCIÓN

1.1 ¿Qué es la capa de percepción?

La capa de percepción es el primer nivel funcional de cualquier sistema IoT. Su función es detectar y medir fenómenos físicos del mundo real — temperatura, presión, humedad, movimiento, precipitación — y convertirlos en señales digitales que un microcontrolador pueda procesar. Sin esta capa, el sistema IoT carece de información válida con la que trabajar (Gubbi et al., 2013).

En la estación meteorológica, la capa de percepción está formada por seis componentes físicos: el módulo dual AHT20+BMP280 (temperatura, humedad y presión), el encoder magnético AS5600 (dirección del viento), dos módulos KY-025 con reed switch (velocidad del viento y precipitación) y la pantalla OLED SSD1306 como salida local de datos. Cada uno opera con un principio físico distinto y se comunica con el ESP32 a través de un protocolo específico.

1.2 Componentes de la capa de percepción

La tabla siguiente presenta todos los componentes que integran la primera capa, su interfaz de comunicación, la variable que miden y el principio físico que utilizan:

Tabla 1 Componentes de la capa de percepción

Componente / Módulo	Interfaz / Dirección	Variable que mide	Principio de operación
AHT20	I2C (0x38) Bus 1	Temperatura + Humedad	Bandgap + capacitivo higroscópico
BMP280	I2C (0x77) Bus 1	Presión + Temperatura	Piezorresistivo con membrana de silicio
AS5600	I2C (0x36) Bus 0	Dirección del viento	Encoder magnético de 12 bits (0–4095)
KY-025 #1	GPIO 34 (interrupción)	Velocidad del viento	Reed switch: pulsos por vuelta (anemómetro)
KY-025 #2	GPIO 35 (interrupción)	Precipitación pluvial	Reed switch: pulsos por volcada (balancín)
SSD1306	I2C (0x3C) Bus 0	Pantalla local	Matriz OLED 128×64 px, protocolo I2C

Los componentes se distribuyen en dos buses I2C independientes y dos pines GPIO con interrupciones de hardware. La separación en dos buses fue una decisión de diseño para evitar interferencias eléctricas entre dispositivos, problema que se detectó durante pruebas iniciales y que se detalla en la sección del BMP280.

1.3 Sensor AHT20 — Temperatura y Humedad Relativa

1.3.1 ¿Qué chip es y qué integra?

El AHT20 es un circuito integrado sensor de temperatura y humedad desarrollado por Aosong Electronics. Se trata de un sensor de precisión industrial de segunda generación que reemplaza al popular DHT11 y DHT22, ofreciendo mayor exactitud y comunicación digital nativa mediante I2C.

En un único encapsulado SMD de aproximadamente $3\text{ mm} \times 3\text{ mm}$ integra los siguientes bloques funcionales:

- Sensor de temperatura tipo bandgap: aprovecha las propiedades eléctricas de los semiconductores para medir temperatura de forma precisa y repetible.
- Sensor capacitivo de humedad relativa: utiliza un polímero higroscópico entre dos electrodos que varía su capacitancia según la cantidad de vapor de agua presente en el ambiente.
- Convertidor analógico-digital (ADC) interno de 20 bits: digitaliza con alta resolución las señales analógicas de ambos sensores.
- Circuito de calibración de fábrica: cada chip almacena internamente coeficientes de compensación individuales que corrigen variaciones de fabricación, garantizando que el sensor funcione correctamente sin calibración externa.
- Interfaz I2C nativa: comunicación a través de dos hilos (SDA y SCL) a la dirección fija 0x38, sin necesidad de componentes externos adicionales.

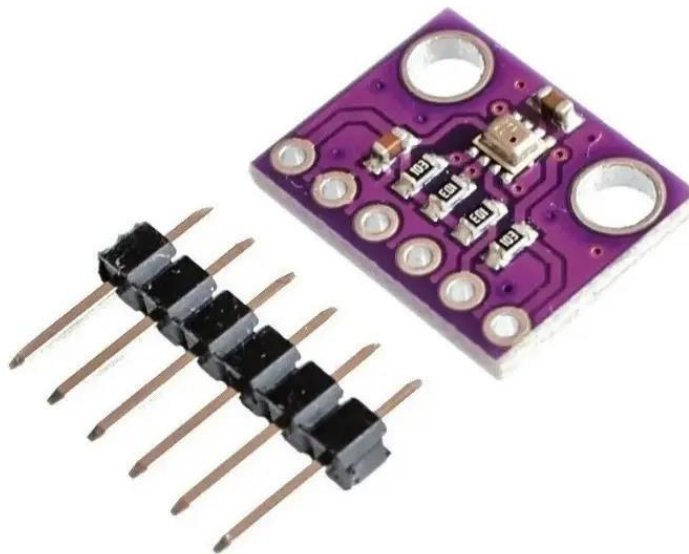


Ilustración 1 Módulo AHT20 / módulo dual AHT20+BMP280

Tabla 2 Especificaciones del módulo AHT20

Parámetro	Especificación
Temperatura — rango	-40 °C a +85 °C
Temperatura — precisión	±0.3 °C
Humedad relativa — rango	0 % a 100 % HR
Humedad relativa — precisión	±2 % HR
Protocolo de comunicación	I2C — dirección 0x38 (Bus 1, GPIO 32/33)
Voltaje de operación	2.0 V a 5.5 V
Resolución	20 bits (temperatura y humedad)
Tiempo de respuesta	~75 ms por medición



Ilustración 2 Stevenson Screen para el módulo BMP280

1.3.2 ¿Cómo mide la temperatura?

El AHT20 utiliza un sensor de banda prohibida (bandgap). Este principio aprovecha una propiedad fundamental de los semiconductores: cuando dos diodos se polarizan con corrientes de diferente magnitud, la diferencia de voltaje entre ellos varía de forma predecible, lineal y proporcional a la temperatura absoluta del semiconductor. Esta relación se conoce como señal PTAT (Proportional To Absolute Temperature — proporcional a la temperatura absoluta).

El circuito interno del AHT20 amplifica esta diferencia de voltaje y la digitaliza con el ADC interno de 20 bits, obteniendo un valor numérico de temperatura con una resolución de 0.01 °C antes de aplicar los coeficientes de calibración de fábrica. El resultado final es un valor de temperatura corregido y listo para usarse, accesible a través del bus I2C sin procesamiento adicional.

1.3.3 ¿Cómo mide la humedad relativa?

Para la humedad relativa, el AHT20 utiliza un sensor capacitivo. El principio de funcionamiento parte de la estructura de un capacitor: dos electrodos metálicos separados por un material

dieléctrico. En el caso del AHT20, ese material es un polímero higroscópico — un material que absorbe moléculas de agua del aire de forma proporcional a la humedad relativa del entorno.

Cuando el vapor de agua penetra en el polímero, su constante dieléctrica cambia. Un cambio en la constante dieléctrica implica directamente un cambio en la capacitancia del sensor, que el circuito de medición interno convierte en un valor digital proporcional al porcentaje de humedad relativa. Los sensores capacitivos de humedad son ampliamente utilizados en instrumentación de precisión por su alta linealidad, buena estabilidad a largo plazo y tiempos de respuesta que oscilan entre 5 y 30 segundos dependiendo del flujo de aire sobre el sensor (Sensirion, 2020).

1.3.4 ¿Cómo se comunica con el ESP32?

El AHT20 utiliza el protocolo I2C a su dirección fija 0x38. En la estación meteorológica está conectado al Bus 1 del ESP32, en los pines GPIO 32 (SDA) y GPIO 33 (SCL), formando un bus aislado junto con el BMP280.

El proceso de medición sigue una secuencia de tres pasos: el ESP32 envía al AHT20 un comando de disparo de medición, espera aproximadamente 75 ms mientras el sensor realiza la conversión interna, y finalmente lee 6 bytes de respuesta que contienen los valores crudos de humedad (20 bits) y temperatura (20 bits) codificados juntos. Estos valores se convierten a unidades físicas mediante las fórmulas de compensación del fabricante, proceso que la librería Adafruit AHTX0 gestiona de forma transparente.

1.4 Sensor BMP280 — Presión Atmosférica y Temperatura

1.4.1 ¿Qué chip es y qué integra?

El BMP280 es un sensor barométrico de alta precisión desarrollado por Bosch Sensortec. Está especialmente diseñado para aplicaciones de consumo, IoT y meteorología donde se requiere medir la presión atmosférica con precisión suficiente para calcular altitud y detectar cambios de clima.

En un encapsulado LGA de 8 pines, el BMP280 integra:

- Elemento piezorresistivo de presión: una membrana microscópica de silicio con cuatro resistencias sensibles a la deformación mecánica.
- Sensor de temperatura interno: para compensar el efecto que la temperatura tiene sobre el elemento de presión.
- ADC de 20 bits: digitaliza las señales de ambos elementos con alta resolución.
- Memoria ROM de fábrica con coeficientes de calibración individuales: cada chip almacena 12 coeficientes únicos que permiten corregir con precisión la lectura de presión y temperatura para ese chip específico.
- Interfaz I2C y SPI: en la estación se utiliza I2C a la dirección 0x77.

- Modos de operación configurables: normal, forced y sleep, con distintas combinaciones de oversampling, filtrado IIR y frecuencia de muestreo para equilibrar resolución y consumo.

Tabla 3 Especificaciones del módulo BMP280

Parámetro	Especificación
Presión — rango	300 hPa a 1100 hPa
Presión — precisión absoluta	±1 hPa
Temperatura — rango	−40 °C a +85 °C
Temperatura — precisión	±0.5 °C
Protocolo de comunicación	I2C — dirección 0x77 (Bus 1, GPIO 32/33)
Voltaje de operación	1.71 V a 3.6 V
Resolución de presión	0.18 Pa
Bus físico en la estación	Wire1 aislado (GPIO 32 SDA / GPIO 33 SCL)

1.4.2 ¿Cómo mide la presión?

El BMP280 basa su funcionamiento en el principio piezorresistivo. Dentro del chip existe una membrana microscópica de silicio fabricada mediante técnicas de microfabricación (MEMS — Micro-Electro-Mechanical Systems). Sobre esta membrana se depositan cuatro resistencias piezorresistivas interconectadas en configuración de puente de Wheatstone.

Cuando la presión atmosférica actúa sobre la membrana, esta se deforma físicamente en la escala de los nanómetros. Esa deformación mecánica modifica la resistencia eléctrica de cada elemento piezorresistivo de forma proporcional a la magnitud de la presión aplicada. El desequilibrio resultante en el puente de Wheatstone genera una diferencia de voltaje que el ADC interno digitaliza. Simultáneamente, el sensor de temperatura interno mide la temperatura del chip y aplica los coeficientes de calibración almacenados en la ROM para compensar el error que la temperatura introduce en la lectura de presión, ya que la deformación mecánica del silicio depende también de la temperatura. Según el fabricante, los sensores de presión piezorresistivos de membrana de silicio son ampliamente utilizados en aplicaciones meteorológicas por su bajo consumo, alta precisión y excelente estabilidad a largo plazo (Bosch Sensortec, 2020).

1.4.3 Problema resuelto: bus I2C aislado

Durante las pruebas iniciales de la estación, el BMP280 devolvía lecturas inválidas (valores de −1000 hPa o fuera del rango físico posible) después de varias horas de operación continua. Tras analizar el problema, la causa identificada fue la interferencia eléctrica producida por otros dispositivos que compartían el mismo bus I2C, en particular la pantalla OLED y el encoder AS5600, que generan ruido en las líneas SDA y SCL.

La solución implementada fue mover el BMP280 y el AHT20 a un bus I2C físicamente independiente, utilizando los pines GPIO 32 (SDA) y GPIO 33 (SCL) del ESP32, que permiten instanciar un segundo bus I2C paralelo. De esta forma, el Bus 0 (GPIO 21/22) queda exclusivamente para el AS5600 y la OLED, y el Bus 1 (GPIO 32/33) para los sensores ambientales. Como medida adicional, el firmware incorpora una función de recuperación automática que reinicia el periférico I2C del ESP32 si detecta una lectura inválida del BMP280, sin necesidad de reiniciar el sistema completo.

1.5 Encoder Magnético AS5600 — Dirección del Viento

1.5.1 ¿Qué chip es y qué integra?

El AS5600 es un encoder magnético de posición absoluta sin contacto físico, desarrollado por ams-OSRAM. A diferencia de los encoders ópticos o mecánicos, el AS5600 no requiere ningún contacto entre el elemento giratorio y el chip detector, lo que lo hace especialmente robusto para aplicaciones en exteriores sujetas a vibración, suciedad, humedad y temperatura variable — condiciones habituales en una veleta meteorológica.

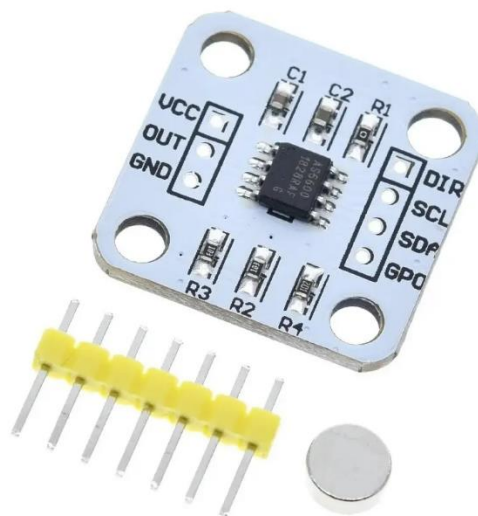


Ilustración 3 Módulo AS5600 con imán de neodimio

En un solo encapsulado, el AS5600 integra:

- Matriz de sensores Hall de 12 bits: detecta la orientación del campo magnético de un imán permanente colocado sobre el eje giratorio de la veleta.
- Procesador de señal digital (DSP): calcula el ángulo exacto a partir de las componentes del campo magnético y aplica compensación automática ante variaciones de temperatura y distancia del imán.

- Interfaz I2C nativa a la dirección fija 0x36.
- Salida analógica DAC y salida PWM, disponibles como alternativas a I2C según la aplicación.
- Registro AGC (Automatic Gain Control): el chip ajusta automáticamente la ganancia de los sensores Hall para compensar variaciones en la intensidad del campo magnético.

Tabla 4 Especificaciones de módulo AS5600

Parámetro	Especificación
Tipo de sensor	Encoder magnético de posición absoluta
Resolución	12 bits — 4096 pasos por vuelta completa (0–4095)
Rango angular	0° a 360° (posición absoluta, sin límite mecánico)
Protocolo de comunicación	I2C — dirección 0x36 (Bus 0, GPIO 21/22)
Voltaje de operación	3.3 V a 5 V
Campo magnético requerido	Imán de neodimio colocado sobre el eje de la veleta
Conversión a grados (firmware)	$\text{ángulo} = \text{rawValue} \times (360.0 / 4096.0)$



Ilustración 4 Veleta con imán para conocer dirección del viento

1.5.2 ¿Cómo mide la dirección del viento?

Sobre el eje de la veleta se coloca un imán de neodimio de forma cilíndrica, diametralmente magnetizado (con el polo norte y el polo sur en lados opuestos del diámetro). El AS5600 se posiciona directamente debajo del imán, a una distancia de entre 0.5 mm y 3 mm, sin contacto físico.

Internamente, la matriz de sensores Hall del AS5600 mide dos componentes ortogonales del campo magnético del imán: la componente horizontal Bx y la componente By. Cuando el imán gira junto con el eje de la veleta, la relación entre estas dos componentes cambia de forma sinusoidal y cosinusoidal. El DSP interno calcula el ángulo de orientación del imán aplicando la función arco tangente (ATAN2) sobre ambas componentes, obteniendo un resultado de 12 bits entre 0 y 4095 que representa la posición angular absoluta del eje dentro de los 360°, sin ambigüedad y sin necesidad de una posición de referencia de arranque.

Este valor de 12 bits se convierte a grados mediante una multiplicación por la constante de conversión $360 / 4096 = 0.0879^\circ$, obteniendo la dirección del viento con una resolución de menos de una décima de grado — considerablemente superior a cualquier veleta convencional de 8 o 16 posiciones. El ángulo obtenido se traduce adicionalmente a punto cardinal (N, NE, E, SE, S, SW, W, NW) para su presentación en la pantalla OLED y en las aplicaciones web y móvil.

1.6 Módulo KY-025 con Reed Switch — Velocidad del Viento (Anemómetro)

1.6.1 ¿Qué es el módulo KY-025 y qué integra?

El KY-025 es un módulo electrónico de detección magnética basado en un reed switch — también llamado interruptor de láminas. Un reed switch es un interruptor electromecánico encapsulado en un tubo de vidrio sellado que contiene dos láminas metálicas ferromagnéticas (generalmente de hierro-níquel). Cuando un campo magnético externo actúa sobre el tubo, las láminas se atraen entre sí y cierran el circuito eléctrico. Al retirarse el campo magnético, las láminas recuperan su posición original por elasticidad mecánica y el circuito se abre.

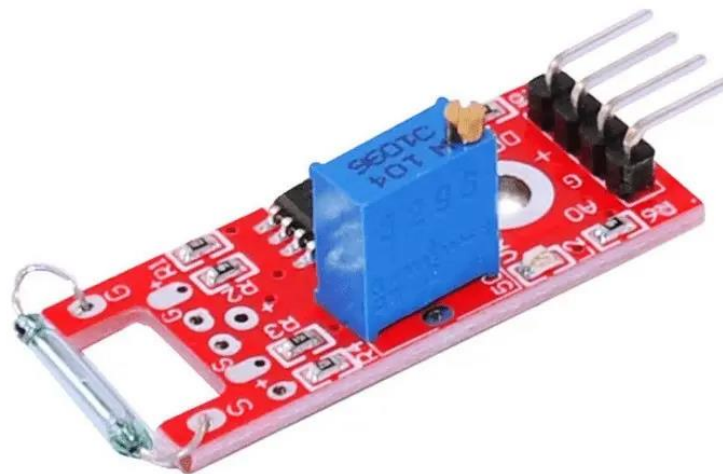


Ilustración 5 Módulo KY-025

El módulo KY-025 integra:

- Reed switch: el elemento sensor de apertura/cierre magnético.
- Resistencia pull-up: mantiene la señal en nivel alto cuando el switch está abierto, evitando que el pin del microcontrolador quede flotante.
- Comparador LM393: convierte la señal del reed switch en una salida digital limpia con histéresis, filtrando el rebote eléctrico (bounce) que producen los contactos mecánicos al abrirse y cerrarse.
- Salida digital (DO) y salida analógica (AO): en la estación se utiliza únicamente la salida digital.
- LED indicador: se ilumina cuando el switch está cerrado, útil para verificación visual durante el montaje.

Tabla 5 Especificaciones del KY-025 para anemómetro

Parámetro	Especificación
Voltaje de operación	3.3 V a 5V
Tipo de sensor	Reed switch (interruptor de láminas magnéticas)
Salida	Digital (DO) — nivel alto en reposo, nivel bajo al activarse
Sensibilidad	Ajustable mediante potenciómetro (umbral del comparador)
Conexión en la estación	GPIO 34 con interrupción de hardware (FALLING)



Ilustración 6 Anemómetro de cazoletas

1.6.2 ¿Cómo mide la velocidad del viento?

El anemómetro utilizado es de tipo cazoletas (cup anemometer), el diseño más común en meteorología. Consiste en tres o cuatro cazoletas semiesféricas montadas sobre brazos horizontales que giran en torno a un eje vertical. El viento entra en las cazoletas abiertas con mayor fuerza que en las cerradas, generando un par neto que hace girar el conjunto a una velocidad de rotación proporcional a la velocidad del viento.

Sobre el eje de rotación del anemómetro se coloca un imán. En cada vuelta completa del eje, el imán pasa frente al reed switch del módulo KY-025 y cierra brevemente el circuito, generando un pulso eléctrico de bajada (FALLING edge). El ESP32 detecta cada uno de estos pulsos mediante una interrupción de hardware en el pin GPIO 34, incrementando un contador en la rutina de servicio de interrupción (ISR).

Cada 5 segundos, el firmware lee y reinicia el contador, calcula el número de vueltas por segundo y aplica la fórmula física de velocidad lineal de la punta del aspa para obtener la velocidad del viento en km/h:

- Vueltas por segundo = pulsos contados / (pulsos por vuelta $\times$ 5 segundos)
- Velocidad en m/s = vueltas por segundo $\times 2\pi \times$ radio del aspa (0.15 m)
- Velocidad en km/h = velocidad en m/s $\times 3.6$

1.7 Módulo KY-025 con Reed Switch — Precipitación Pluvial (Pluviómetro)

1.7.1 ¿Qué es el pluviómetro de balancín y cómo funciona?

El pluviómetro utilizado es de tipo balancín (tipping bucket rain gauge), el diseño estándar en estaciones meteorológicas automáticas. Consiste en un embudo colector que canaliza el agua de lluvia hacia un balancín bicompartimental de precisión. Cada compartimento tiene una capacidad calibrada de exactamente 2.5 mL (equivalente a 0.25 mm de lluvia sobre el área del embudo de 100 cm²). Cuando un compartimento se llena, el peso del agua vence la tensión del resorte central y el balancín vuelca hacia ese lado, vaciando el compartimento lleno y posicionando el compartimento vacío bajo el chorro de agua para iniciar el siguiente ciclo.

En la parte inferior del balancín hay un imán permanente. Cada vez que el balancín vuelca (en cualquier dirección), el imán pasa frente al reed switch del módulo KY-025 conectado al GPIO 35, generando un pulso eléctrico. El ESP32 cuenta cada pulso mediante una interrupción de hardware y acumula la precipitación total multiplicando el número de pulsos por el volumen calibrado de cada volcada, dividido entre el área del embudo colector.



Ilustración 7 Pluviómetro

1.7.2 Cálculo de la precipitación acumulada

La precipitación pluvial se expresa en milímetros (mm), que equivale al volumen de agua caído por unidad de área: 1 mm = 1 litro por metro cuadrado. La conversión de pulsos del balancín a milímetros de lluvia se realiza con la siguiente relación física:

- Volumen por volcada = 2.5 mL
- Área del embudo colector = 100 cm²
- Precipitación por volcada = $(2.5 \text{ mL} / 100 \text{ cm}^2) \times 10 = 0.25 \text{ mm por pulso}$

Esta acumulación se registra en tiempo real y se reinicia automáticamente al inicio de cada registro histórico de 10 minutos, permitiendo calcular tanto la lluvia acumulada total como la intensidad de precipitación por periodo.

1.8 Pantalla OLED SSD1306 — Visualización Local

1.8.1 ¿Qué es el SSD1306 y qué integra?

El SSD1306 es un controlador de pantalla OLED (Organic Light-Emitting Diode) monocromático desarrollado por Solomon Systech. En la estación meteorológica se utiliza en un módulo de 0.96 pulgadas con resolución de 128×64 píxeles que actúa como salida local de datos, permitiendo leer los valores meteorológicos directamente en la estación sin necesidad de una red WiFi activa ni de un teléfono o computadora.



Ilustración 8 Pantalla OLED SSD1306

El módulo integra:

- Panel OLED 128×64 píxeles: cada píxel es un diodo orgánico que emite luz propia al recibir corriente, sin necesidad de retroiluminación. Esto permite que el display sea delgado, de bajo consumo y con excelente contraste y ángulo de visión.
- Controlador SSD1306: gestiona la memoria de video (GDDRAM — Graphic Display Data RAM) de 1 KB que almacena el estado encendido/apagado de cada uno de los 8,192 píxeles de la pantalla.
- Convertidor de carga integrado (charge pump): genera internamente el voltaje de panel (~7-9 V) necesario para polarizar los píxeles OLED a partir de la alimentación de 3.3 V del ESP32, sin requerir fuente de voltaje adicional.
- Interfaz I2C a la dirección 0x3C, compartiendo el Bus 0 (GPIO 21/22) con el encoder AS5600.

Tabla 6 Especificaciones de la pantalla OLED SSD1306

Parámetro	Especificación
Resolución	128 × 64 píxeles
Tipo de display	OLED monocromático (blanco sobre negro)
Voltaje de operación	3.3 V a 5 V
Protocolo de comunicación	I2C — dirección 0x3C (Bus 0, GPIO 21/22)
Consumo típico	~20 mA con pantalla completa activa
Diagonal visible	0.96 pulgadas

1.8.2 Funcionamiento del display en la estación

La pantalla muestra cuatro pantallas rotativas que se alternan automáticamente cada 4 segundos, presentando toda la información de la estación en grupos temáticos:

- Pantalla 1 — Temperatura y Humedad: muestra la temperatura en grande con indicador contextual (temperatura agradable / hace calor / calor extremo / hace frío) y el porcentaje de humedad relativa.
- Pantalla 2 — Presión y Viento: muestra la presión atmosférica en hPa, la velocidad del viento en km/h y la dirección en punto cardinal.
- Pantalla 3 — Lluvia y Estado: muestra la precipitación acumulada en mm y el estado de conexión WiFi y Firebase.
- Pantalla 4 — Red y Bluetooth: muestra el nombre de la red WiFi conectada, la dirección IP local, el estado del módulo BLE y la intensidad de señal WiFi en dBm.

1.9 Decisión de diseño: dos buses I2C independientes

Una de las decisiones de diseño más importantes de la capa de percepción fue la separación de los dispositivos I2C en dos buses físicamente independientes. El protocolo I2C permite conectar múltiples dispositivos en el mismo par de cables (SDA y SCL) siempre que cada uno tenga una dirección única. Sin embargo, cuando un dispositivo presenta problemas eléctricos — como reiniciarse, quedar en un estado de bloqueo o generar ruido en las líneas — puede interferir con las comunicaciones de todos los demás dispositivos del mismo bus.

Durante las pruebas de funcionamiento continuo se detectó que el BMP280 generaba lecturas completamente inválidas (valores fuera del rango físico posible) tras varias horas de operación. La causa fue interferencia eléctrica producida por la pantalla OLED SSD1306 y el encoder AS5600 en el mismo bus, que generaban condiciones eléctricas transitorias incompatibles con el protocolo I2C en ciertos momentos.

La solución implementada fue asignar los sensores ambientales (AHT20 y BMP280) a un segundo bus I2C físico usando los pines GPIO 32 y 33 del ESP32, que el framework Arduino permite instanciar como un objeto `Wire1` independiente del `Wire` principal. Esta separación garantiza que

los eventos eléctricos de la pantalla OLED o el encoder no afecten en ningún caso las lecturas de temperatura, humedad y presión.

Tabla 7 Buses I2C independientes

Bus	Pines ESP32	Dispositivos
Bus 0 — Wire	GPIO 21 (SDA) / GPIO 22 (SCL)	AS5600 (0x36) + SSD1306 (0x3C)
Bus 1 — Wire1	GPIO 32 (SDA) / GPIO 33 (SCL)	AHT20 (0x38) + BMP280 (0x77)

2. SEGUNDA CAPA: RED

2.1 ¿Qué hace esta capa?

La capa de red es el puente entre el mundo físico y el mundo digital. Su función es tomar los datos que los sensores capturaron y el microcontrolador procesó, y transportarlos de forma confiable hasta el servidor en la nube, y desde ahí hasta los dispositivos del usuario final. Sin esta capa, los datos quedarían atrapados en el ESP32 sin posibilidad de ser almacenados, visualizados ni compartidos. La capa de red determina la latencia, el consumo energético y la confiabilidad con que los datos llegan al servidor — factores críticos en aplicaciones de monitoreo en tiempo real (Al-Fuqaha et al., 2015).

En la estación meteorológica, esta capa cubre dos caminos de comunicación distintos:

- Camino principal: ESP32 → WiFi → Internet → Firebase Realtime Database → App web / App Flutter
- Camino de configuración: Teléfono móvil → BLE → ESP32 (para enviar credenciales WiFi sin recompilar código)

2.2 Tecnología de red principal — WiFi 802.11n

2.2.1 ¿Qué chip maneja el WiFi en el ESP32?

El ESP32 no necesita ningún módulo WiFi externo. El chip ESP32 (Espressif Systems) integra en un solo SoC (System on Chip) el procesador dual-core y un transceptor de radiofrecuencia completo compatible con el estándar IEEE 802.11 b/g/n, operando en la banda de 2.4 GHz. El transceptor incluye amplificador de potencia de transmisión (PA), amplificador de bajo ruido de recepción (LNA), filtros de radiofrecuencia y una antena de traza PCB integrada en la placa del módulo.

Esto significa que el WiFi no se comunica a través de ningún pin físico hacia el exterior — es un periférico interno del chip, igual que el ADC o los buses I2C. La antena impresa en la PCB del módulo ESP32-WROOM-32 es el único elemento físico visible relacionado con WiFi, y se identifica como la lengüeta metálica que sobresale en un extremo de la placa.

2.2.2 Especificaciones del módulo WiFi integrado

Tabla 8 Especificaciones del WIFI integrado

Parámetro	Especificación
Estándar	IEEE 802.11 b/g/n
Banda de frecuencia	2.4 GHz
Velocidad máxima de transferencia	150 Mbps (802.11n, una antena)
Potencia de transmisión máxima	19.5 dBm
Modos de operación	Station (STA), Access Point (AP), STA+AP simultáneo
Seguridad soportada	WPA / WPA2 / WPA3
Antena	Traza PCB integrada en el módulo ESP32-WROOM-32
Asignación de IP	DHCP automático o IP estática configurable

2.2.3 ¿Cómo establece la conexión el ESP32?

El proceso de conexión WiFi ocurre automáticamente al arrancar el ESP32 y sigue estos pasos en orden:

Paso 1 — Lectura de credenciales desde memoria flash. El ESP32 lee el nombre de red (SSID) y la contraseña almacenados previamente en su memoria flash no volátil usando la librería Preferences. Si no hay credenciales guardadas, el sistema queda en espera de configuración por BLE.

Paso 2 — Asociación al router. El ESP32 escanea las redes disponibles en 2.4 GHz y se asocia al router usando las credenciales leídas, realizando el handshake de seguridad WPA2.

Paso 3 — Asignación de dirección IP por DHCP. Una vez asociado, el router le asigna automáticamente una dirección IP local mediante el protocolo DHCP (Dynamic Host Configuration Protocol), permitiendo al ESP32 comunicarse dentro de la red local y acceder a internet a través del router.

Paso 4 — Sincronización de tiempo con NTP. Con acceso a internet disponible, el ESP32 consulta al servidor pool.ntp.org mediante el protocolo NTP (Network Time Protocol) para sincronizar su reloj interno con la hora real. Esto es indispensable para generar los timestamps correctos en cada registro meteorológico.

Paso 5 — Conexión a Firebase. Finalmente, el ESP32 establece la sesión autenticada con Firebase Realtime Database usando el database secret del proyecto, habilitando el envío de datos.

2.3 Protocolo de transporte — HTTPS sobre TCP/IP

2.3.1 Modelo de capas de la comunicación ESP32 → Firebase

La comunicación desde el ESP32 hasta Firebase sigue el modelo de red por capas estándar. Cada capa tiene una responsabilidad específica y las capas superiores se apoyan en las inferiores:

Tabla 9 Modelo de capas y su protocolo utilizado

Capa del modelo	Protocolo utilizado	Función en la estación
Capa física y enlace	WiFi 802.11n (2.4 GHz)	Transmisión de ondas de radio entre ESP32 y router
Capa de red	IP versión 4 (IPv4)	Enrutamiento de paquetes entre ESP32, router e internet
Capa de transporte	TCP (Transmission Control Protocol)	Entrega ordenada, sin errores y sin pérdida de paquetes
Capa de aplicación	HTTPS + TLS 1.2	Cifrado de la comunicación y transporte de datos JSON hacia Firebase

2.3.2 ¿Por qué HTTPS y no HTTP?

Firebase requiere HTTPS de forma obligatoria. HTTPS (HyperText Transfer Protocol Secure) es HTTP con una capa de cifrado TLS (Transport Layer Security) que protege la comunicación en ambas direcciones. TLS utiliza cifrado asimétrico durante el establecimiento de la conexión (para intercambiar claves de forma segura) y cifrado simétrico durante la transferencia de datos (para proteger el contenido). Esto garantiza que las credenciales de Firebase y los datos meteorológicos no puedan ser interceptados ni modificados en tránsito, incluso en redes WiFi compartidas o públicas (Rescorla, 2018).

El ESP32 gestiona todo el handshake TLS internamente a través de la librería mbedTLS, incluida en el core ESP32 de Espressif, sin que el firmware de la estación necesite implementar ninguna parte del proceso criptográfico manualmente.

2.4 Estructura del flujo de datos hacia Firebase

2.4.1 Serialización en formato JSON

Antes de enviarse por la red, todos los valores de los sensores se empaquetan en un objeto JSON (JavaScript Object Notation). JSON es el formato estándar de intercambio de datos en aplicaciones web y APIs REST por su legibilidad humana, compatibilidad universal y bajo overhead frente a formatos binarios alternativos en aplicaciones de baja frecuencia de envío (Google, 2023).

Cada paquete de datos que el ESP32 construye y envía a Firebase tiene la siguiente estructura:

```
{
  "timestamp":    "2026-03-19T14:32:05",
  "temperatura":  28.4,
  "humedad":      65.2,
  "presion":      893.7,
  "viento_kmh":   12.6,
  "viento_dir":   "NE",
  "viento_grados": 47.3,
  "lluvia_mm":    0.0
}
```

Ilustración 9 Estructura del archivo JSON que se envía a Firebase

2.4.2 Estrategia de envío — dos canales con distinta frecuencia

Una decisión clave de diseño en esta capa es la separación del envío en dos canales con frecuencias distintas, cada uno con un propósito específico:

Tabla 10 Estrategia de envío en dos canales con distinta frecuencia

Canal	Nodo en Firebase	Frecuencia	Comportamiento	Propósito
Tiempo real	/estacion/actual	Cada 1.5 segundos	Sobreescribe siempre el mismo documento	Alimentar el dashboard en vivo de la app web y Flutter
Histórico	/estacion/historial/{timestamp}	Cada 10 minutos	Crea un documento nuevo con el timestamp como clave	Alimentar gráficas y tabla de registros

Esta separación evita que la base de datos crezca de forma descontrolada: el canal de tiempo real siempre ocupa el espacio de un solo documento, mientras el canal histórico genera aproximadamente 144 registros por día y 52,560 por año — una cantidad eficiente y manejable dentro del plan gratuito Spark de Firebase.

2.5 Recepción de datos en el celular — Firebase SDK

2.5.1 ¿Cómo llegan los datos al teléfono?

Una vez que el ESP32 escribe un nuevo dato en Firebase, tanto la aplicación web como la aplicación Flutter del celular lo reciben automáticamente sin necesidad de hacer consultas periódicas. Esto es posible gracias al mecanismo de escucha en tiempo real (listener) que ofrece Firebase Realtime Database.

Firebase mantiene una conexión WebSocket persistente con cada cliente conectado (el navegador o la app Flutter). Cuando el ESP32 escribe un nuevo valor en /estacion/actual, Firebase notifica instantáneamente a todos los clientes suscritos a ese nodo, que reciben el nuevo dato en milisegundos. Este modelo de publicación-suscripción elimina la necesidad de que el celular consulte repetidamente al servidor — es Firebase quien empuja los datos hacia el cliente cada vez que cambian.

2.5.2 Protocolo de comunicación Firebase → celular

Tabla 11 Protocolo de comunicación de Firebase a Movil

Tramo	Protocolo	Descripción
Firebase → Navegador (app web)	WebSocket sobre HTTPS	Conexión persistente gestionada por el Firebase Web SDK v10
Firebase → App Flutter (Android)	WebSocket sobre HTTPS	Conexión persistente gestionada por el paquete firebase_database
Firebase → Notificaciones push	FCM (Firebase Cloud Messaging)	Mensajes push enviados al celular cuando se superan umbrales de alerta.

2.6 Configuración WiFi por BLE — protocolo del celular al ESP32

2.6.1 ¿Para qué sirve BLE en esta capa?

El ESP32 necesita conocer el nombre (SSID) y la contraseña de la red WiFi a la que debe conectarse. Hardcodear estas credenciales dentro del código fuente tiene un problema práctico importante: si la red WiFi cambia, habría que recompilar y re-flashear el ESP32. Para evitar esto, la estación implementa un canal de configuración alternativo usando BLE (Bluetooth Low

Energy)), que permite enviar las credenciales WiFi directamente desde el teléfono al ESP32 sin necesidad de cables, computadora ni acceso a internet.

2.6.2 ¿Qué pin usa el BLE en el ESP32?

Al igual que el WiFi, el BLE no utiliza ningún pin físico externo. El ESP32 integra en el mismo SoC un transceptor Bluetooth 4.2 (BLE + Bluetooth clásico) que comparte la misma antena de traza PCB que el WiFi. El BLE es un periférico interno del chip, gestionado completamente por software a través del stack BLE del ESP-IDF incluido en el core de Espressif.

2.6.3 Especificaciones del módulo BLE integrado

Tabla 12 Especificaciones del módulo BLE

Parámetro	Especificación
Estándar	Bluetooth 4.2 (BLE — Bluetooth Low Energy)
Banda de frecuencia	2.4 GHz (canales de 2 MHz entre 2402 y 2480 MHz)
Perfil utilizado	GATT (Generic Attribute Profile) — servidor de características
Nombre visible desde el celular	EstacionMeteo
Antena	Traza PCB integrada, compartida con el módulo WiFi
Alcance típico	10–30 metros en espacio abierto

2.6.4 ¿Cómo funciona la comunicación BLE entre el celular y el ESP32?

BLE organiza la comunicación en torno al perfil GATT (Generic Attribute Profile). En este modelo, el ESP32 actúa como servidor GATT y el teléfono actúa como cliente GATT. El servidor expone un servicio con características — equivalentes a campos de una base de datos — que el cliente puede leer, escribir o suscribirse para recibir notificaciones.

La estación meteorológica define un servicio BLE con tres características:

Tabla 13 Características del servicio BLE implementadas

Característica	UUID	Propiedad	Función
SSID	beb5483e-...-26a8	WRITE	El celular escribe el nombre de la red WiFi
Contraseña	beb5483e-...-26a9	WRITE	El celular escribe la contraseña WiFi
Estado	beb5483e-...-26aa	READ + NOTIFY	El ESP32 notifica al celular el resultado de la conexión

El flujo de configuración completo funciona así:

Paso 1 — Advertising. El ESP32 transmite continuamente paquetes de advertising BLE con su nombre "EstacionMeteo", permitiendo que cualquier dispositivo cercano lo descubra.

Paso 2 — Conexión. El usuario abre la pantalla WifiConfigScreen de la app Flutter, que escanea dispositivos BLE cercanos y se conecta a "EstacionMeteo" usando la librería flutter_blue_plus.

Paso 3 — Escritura de credenciales. La app escribe el SSID en la característica correspondiente y luego escribe la contraseña. Cuando el ESP32 recibe la contraseña, sabe que tiene ambas credenciales completas y activa el proceso de conexión WiFi.

Paso 4 — Notificación de resultado. El ESP32 intenta conectarse al router con las credenciales recibidas. El resultado (éxito con la IP asignada, o error de conexión) se notifica de vuelta al celular a través de la característica de Estado, cuyo valor se actualiza y se envía mediante notify hacia la app.

Paso 5 — Persistencia. Si la conexión WiFi fue exitosa, el ESP32 guarda las credenciales en su memoria flash no volátil usando la librería Preferences, de modo que al reiniciarse las recupera automáticamente sin necesidad de configurar de nuevo por BLE.

2.7 Tolerancia a fallos en la capa de red

La capa de red implementa tres mecanismos de recuperación automática ante fallos de conectividad:

Tabla 14 Mecanismos de recuperación automática

Fallo	Mecanismo de recuperación	Resultado
Pérdida de señal WiFi	Firestore.reconnectWiFi(true) — la librería FirestoreESP32 reintenta la conexión automáticamente	El ESP32 se reconecta sin reiniciarse cuando el router vuelve a estar disponible
Firestore temporalmente no disponible	El ESP32 continúa el loop leyendo sensores y reintenta el envío en el siguiente ciclo (cada 1.5 s)	No se interrumpe el funcionamiento local; se pierde solo el envío del ciclo afectado
Credenciales WiFi incorrectas o red no encontrada	El intento de conexión agota 20 reintentos con timeout de 500 ms cada uno; se notifica el error por BLE si hay un celular conectado	El sistema continúa funcionando localmente (OLED + Serial) y queda disponible para nueva configuración por BLE

3. TERCERA CAPA: PROCESAMIENTO

3.1 ¿Qué hace esta capa?

La capa de procesamiento es responsable de recibir las señales crudas de los sensores, interpretarlas, aplicar las operaciones matemáticas necesarias para convertirlas en valores físicos reales, validarlas, organizarlas y prepararlas para su envío a la nube. Sin esta capa, los sensores solo producen números sin significado — un valor de 4096 proveniente del AS5600 no dice nada por sí solo hasta que se multiplica por la constante de conversión y se traduce a grados y punto cardinal. En la arquitectura IoT, la capa de procesamiento filtra, transforma y prepara los datos crudos provenientes de los sensores antes de enviarlos a capas superiores, reduciendo el volumen de información transmitida y el consumo de ancho de banda (Zaslavsky et al., 2013).

En la estación meteorológica, esta capa está representada completamente por el microcontrolador ESP32.

3.2 El microcontrolador ESP32 — hardware de procesamiento

3.2.1 ¿Qué es el ESP32 y qué integra?

El ESP32 es un SoC (System on Chip) desarrollado por Espressif Systems que integra en un único componente todo lo necesario para operar como nodo IoT autónomo: procesador, memoria, periféricos de entrada/salida y módulos de comunicación inalámbrica. A diferencia de un microprocesador convencional que requiere componentes externos para funcionar, el ESP32 contiene todo integrado y está diseñado para ejecutar una tarea específica de forma continua y eficiente, operando 24 horas al día los 7 días de la semana (Espressif Systems, 2023).



Ilustración 10 Placa de desarrollo ESP32

Tabla 15 Especificaciones del ESP32

Característica	Especificación
Procesador	Xtensa LX6 dual-core de 32 bits
Velocidad de reloj	Hasta 240 MHz
RAM	520 KB SRAM
Memoria flash	4 MB (almacenamiento del programa)
WiFi	802.11 b/g/n integrado — 2.4 GHz
Bluetooth	BLE 4.2 + Bluetooth clásico integrado
Pines GPIO	34 pines configurables
Protocolos soportados	I2C, SPI, UART, PWM, ADC, DAC
Voltaje de operación	3.3 V (algunos pines toleran 5 V)
Partición de memoria usada	Huge APP (3 MB) — requerida por BLE + Firebase simultáneos

3.2.2 Mapa de pines — conexiones físicas del ESP32

Cada sensor y periférico de la estación se conecta al ESP32 a través de pines específicos. La tabla siguiente muestra el mapa completo de conexiones físicas:

Tabla 16 Conexiones físicas del ESP32

Pin GPIO	Función asignada	Dispositivo conectado	Dirección
GPIO 21	SDA — Bus I2C 0	AS5600 (encoder) + SSD1306 (OLED)	Bidireccional
GPIO 22	SCL — Bus I2C 0	AS5600 (encoder) + SSD1306 (OLED)	Salida (reloj)
GPIO 32	SDA — Bus I2C 1	AHT20 (temp/hum) + BMP280 (presión)	Bidireccional
GPIO 33	SCL — Bus I2C 1	AHT20 (temp/hum) + BMP280 (presión)	Salida (reloj)
GPIO 34	Entrada digital con interrupción	KY-025 #1 — anemómetro (velocidad viento)	Entrada
GPIO 35	Entrada digital con interrupción	KY-025 #2 — pluviómetro (lluvia)	Entrada
Interno	Transceptor WiFi/BLE	Antena PCB integrada	Bidireccional inalámbrico

3.3 Declaración de variables globales

Antes de que el programa inicie, el firmware declara todas las variables que almacenan el estado del sistema durante la ejecución. Las variables se organizan por categoría según su función:

3.3.1 Variables de conteo de pulsos — declaradas como volatile

Los pines GPIO 34 y 35 generan interrupciones de hardware cada vez que el anemómetro o el pluviómetro producen un pulso. Las variables que almacenan estos conteos se declaran con el modificador volatile, que le indica al compilador que el valor puede cambiar en cualquier momento desde fuera del flujo normal del programa (desde la rutina de interrupción), y que por lo tanto no debe optimizarlas ni cachearlas en registros del procesador:

Tabla 17 Variables globales de conteo

Variable	Tipo	Valor inicial	Descripción
contViento	volatile int	0	Cuenta pulsos del anemómetro entre cada medición de 5 s
contLluvia	volatile int	0	Cuenta pulsos del pluviómetro desde el inicio o último reset

3.3.2 Variables de resultado meteorológico

Tabla 18 Variables de resultado

Variable	Tipo	Unidad	Descripción
velocidadViento	float	km/h	Velocidad calculada del viento en el último intervalo de 5 s
lluviaAcumulada	float	mm	Precipitación acumulada desde el último registro histórico
temp	float	°C	Temperatura leída del AHT20 en el ciclo actual
humedad	float	% HR	Humedad relativa leída del AHT20 en el ciclo actual
presion	float	hPa	Presión atmosférica leída del BMP280 en el ciclo actual

angulo	float	grados (0–360°)	Ángulo de la veleta leído del AS5600 en el ciclo actual
dir	String	—	Punto cardinal calculado a partir del ángulo
ts	String	—	Timestamp ISO 8601 generado con la hora NTP actual

3.3.3 Variables de control de tiempo

El firmware no utiliza delay() para controlar los intervalos de envío y actualización, ya que delay() bloquea completamente el procesador e impediría contar pulsos de interrupción y actualizar la pantalla simultáneamente. En su lugar, se usa el patrón de temporización no bloqueante con millis(), que registra el tiempo transcurrido desde el arranque:

Tabla 19 Variables de control de tiempo

Variable	Tipo	Intervalo asociado	Descripción
tAnteriorViento	unsigned long	5,000 ms	Marca de tiempo de la última medición de velocidad del viento
tAnteriorConsola	unsigned long	2,000 ms	Marca de tiempo de la última impresión en el Serial Monitor
tAnteriorOLED	unsigned long	4,000 ms	Marca de tiempo del último cambio de pantalla en la OLED
tAnteriorActual	unsigned long	1,500 ms	Marca de tiempo del último envío al nodo /estacion/actual
tAnteriorHist	unsigned long	600,000 ms	Marca de tiempo del último registro guardado en el historial

3.3.4 Variables de estado del Sistema

Tabla 20 Variables de estado del sistema

Variable	Tipo	Descripción
pantallaActual	int	Índice (0–3) de la pantalla OLED actualmente visible
bleActivo	bool	Indica si el módulo BLE fue inicializado correctamente
wifiNuevo	bool	Flag que se activa cuando BLE recibe nuevas credenciales WiFi
nuevaSSID	String	SSID recibido por BLE, pendiente de aplicar
nuevaPass	String	Contraseña recibida por BLE, pendiente de aplicar
fbOK	bool	Resultado del último intento de escritura en Firebase

3.4 Rutinas de interrupción (ISR)

Las interrupciones de hardware son el mecanismo que permite al ESP32 reaccionar instantáneamente a eventos externos — como el cierre de un reed switch — sin importar qué otra tarea esté ejecutando en ese momento. Cuando el anemómetro o el pluviómetro generan un pulso, el procesador suspende temporalmente el loop principal, ejecuta la rutina de interrupción y retoma exactamente donde lo dejó. Esto garantiza que ningún pulso se pierda, incluso si el ESP32 está en ese instante enviando datos a Firebase o actualizando la pantalla OLED.

Las rutinas de servicio de interrupción (ISR) se declaran con el atributo `IRAM_ATTR`, que le indica al compilador que debe almacenar esa función en la RAM interna del ESP32 (IRAM) y no en la memoria flash. Esto es obligatorio porque el acceso a flash es considerablemente más lento que el acceso a RAM, y una ISR debe ejecutarse en el menor tiempo posible para no interrumpir el sistema por más tiempo del necesario:

Tabla 21 Rutinas de servicio de interrupción

ISR	Pin	Flanco detectado	Acción
isrViento()	GPIO 34	FALLING (bajada)	Incrementa contViento en 1
isrLluvia()	GPIO 35	FALLING (bajada)	Incrementa contLluvia en 1

El flanco FALLING (de nivel alto a nivel bajo) se detecta porque el módulo KY-025 mantiene la salida en nivel alto en reposo gracias a su resistencia pull-up interna, y la lleva a nivel bajo cuando el reed switch se cierra al pasar el imán.

3.5 Operaciones matemáticas de procesamiento

3.5.1 Conversión de pulsos a velocidad del viento

La velocidad del viento se calcula cada 5 segundos a partir del número de pulsos acumulados por la ISR del anemómetro. El procesamiento sigue tres pasos matemáticos:

Paso 1 — Vueltas por segundo:

$$\text{vueltas por segundo} = \frac{\text{pulsos contados}}{\text{pulsos por vuelta} \times 5 \text{ s}}$$

Con 1 pulso por vuelta configurado:

$$\text{vueltas por segundo} = \frac{\text{contViento}}{1 \times 5} = \frac{\text{contViento}}{5}$$

Paso 2 — Velocidad lineal de la punta del aspa en m/s:

La punta del aspa recorre la circunferencia completa del círculo que describe al girar. Con un radio de aspa de 0.15 m:

$$\text{velocidad (m/s)} = \text{vueltas por segundo} \times 2\pi \times 0.15$$

Paso 3 — Conversión a km/h:

$$\text{velocidad (km/h)} = \text{velocidad (m/s)} \times 3.6$$

Para proteger el valor durante la lectura del contador (que puede ser modificado por la ISR en cualquier momento), el firmware deshabilita temporalmente las interrupciones, copia el valor y lo reinicia, y luego las reactiva:

```
noInterrupts();  
int pv = contViento;  
contViento = 0;  
interrupts();
```

Ilustración 11 Protección del valor durante la lectura del contador

3.5.2 Conversión de pulsos a precipitación en milímetros

Cada volcada del pluviómetro equivale a 2.5 mL de agua recogida sobre un embudo de 100 cm². La conversión a milímetros de lluvia aplica la relación entre volumen y área:

$$\text{lluvia por pulso (mm)} = \frac{\text{ML_POR_PULSO}}{\text{AREA_EMBUDO_CM}^2} \times 10 = \frac{2.5}{100} \times 10 = 0.25 \text{ mm}$$

La precipitación acumulada se actualiza en cada ciclo del loop:

$$\text{lluviaAcumulada} += \text{pulsos} \times 0.25 \text{ mm}$$

La variable lluviaAcumulada se reinicia a cero después de cada registro histórico (cada 10 minutos), permitiendo calcular la precipitación por periodo además de la acumulada total.

3.5.3 Conversión del valor crudo del AS5600 a grados y punto cardinal

El AS5600 entrega un entero de 12 bits entre 0 y 4095 que representa la posición angular completa de 360°. La conversión a grados aplica una regla de tres directa usando la constante AS5600_RAW_TO_DEGREES = 360.0 / 4096.0:

$$\text{ángulo (}^\circ\text{)} = \text{valor crudo} \times \frac{360}{4096} = \text{valor crudo} \times 0.08789^\circ$$

A partir del ángulo en grados se determina el punto cardinal dividiendo los 360° en 8 sectores de 45° cada uno, centrados en cada dirección:

Tabla 22 Rango de ángulo para determinar punto cardinal

Rango de ángulo	Punto cardinal
0° a 22.4° y 337.5° a 360°	N — Norte
22.5° a 67.4°	NE — Noreste
67.5° a 112.4°	E — Este
112.5° a 157.4°	SE — Sureste
157.5° a 202.4°	S — Sur
202.5° a 247.4°	SW — Suroeste
247.5° a 292.4°	W — Oeste
292.5° a 337.4°	NW — Noroeste

3.5.4 Conversión de la presión cruda del BMP280 a hPa

El BMP280 entrega la presión en Pascales (Pa) como número entero sin signo de 32 bits, ya con la compensación de temperatura aplicada internamente por la librería. La conversión a hectopascales (hPa), la unidad estándar en meteorología, es una simple división entre 100:

$$\text{presión (hPa)} = \frac{\text{presión (Pa)}}{100}$$

Antes de realizar esta conversión, el firmware valida que el valor crudo sea físicamente posible. Una lectura se considera inválida si es NaN (Not a Number), infinita, menor o igual a cero, o mayor a 120,000 Pa (equivalente a 1,200 hPa, por encima del máximo físico posible en la Tierra). Si el valor es inválido, se activa la recuperación automática del bus I2C.

3.6 Validación de datos y recuperación automática

3.6.1 Función de validación

Antes de enviar cualquier dato a Firebase o mostrarlo en la OLED, el firmware valida cada valor con la función `esValido()`. Esta función descarta valores que el lenguaje C++ marca como NaN o infinito — resultados que pueden ocurrir cuando un sensor no responde o devuelve un registro de error:

La condición de envío a Firebase requiere que todos los valores pasen la validación simultáneamente. Si cualquiera falla, el ciclo de envío se omite y se registra en el Serial Monitor el motivo:

Tabla 23 Validaciones de datos

Condición verificada	Descripción
<code>esValido(temp)</code>	Temperatura no es NaN ni infinito
<code>esValido(humedad)</code>	Humedad no es NaN ni infinito
<code>esValido(presion) && presion > 0</code>	Presión válida y físicamente posible
<code>esValido(velocidadViento)</code>	Velocidad del viento no es NaN ni infinito
<code>esValido(angulo)</code>	Ángulo del encoder no es NaN ni infinito

3.6.2 Recuperación automática del bus I2C

Si el BMP280 devuelve una lectura inválida, el firmware ejecuta la función `resetBusI2C()` que reinicia ambos buses I2C del ESP32 sin necesidad de reiniciar el sistema completo. El proceso de recuperación sigue estos pasos:

Tabla 24 Pasos para recuperar el bus I2C automáticamente

Paso	Bus 0 (Wire — GPIO 21/22)	Bus 1 (Wire1 — GPIO 32/33)
1	<code>Wire.end()</code> — libera el periférico	<code>Wire1.end()</code> — libera el periférico
2	Espera 200 ms para estabilización eléctrica	Espera 200 ms para estabilización eléctrica
3	<code>Wire.begin(21, 22)</code> — reinicia el bus	<code>Wire1.begin(32, 33)</code> — reinicia el bus
4	Reinicia AS5600 y OLED SSD1306	Reinicia AHT20 y BMP280 con configuración de muestreo

3.7 Estructura del programa — `setup()` y `loop()`

El firmware del ESP32 sigue la estructura estándar del framework Arduino, compuesta por dos funciones principales:

3.7.1 `setup()` — inicialización única al arranque

El `setup()` se ejecuta una sola vez cuando el ESP32 enciende o se reinicia. Realiza en orden las siguientes operaciones:

Tabla 25 Estructura del programa: `setup()`

Orden	Operación	Descripción
1	<code>Serial.begin(115200)</code>	Abre el puerto serie para el Serial Monitor de depuración
2	<code>Wire.begin(21, 22)</code>	Inicializa el Bus I2C 0
3	<code>Wire1.begin(32, 33)</code>	Inicializa el Bus I2C 1
4	<code>initOLED()</code>	Inicializa la pantalla OLED y muestra la pantalla de bienvenida
5	<code>iniciarBLE()</code>	Inicia el servidor GATT BLE con nombre "EstacionMeteo"
6	<code>leerSSID()</code> / <code>leerPass()</code>	Lee credenciales WiFi guardadas en flash
7	<code>conectarWifi()</code>	Intenta conectar al router con las credenciales leídas

8	<code>aht.begin() / bmp.begin()</code>	Inicializa los sensores del Bus 1
9	<code>encoder.begin()</code>	Inicializa el AS5600 en el Bus 0
10	<code>pinMode() + attachInterrupt()</code>	Configura GPIO 34 y 35 como entradas con ISR en flanco FALLING
11	<code>configTime()</code>	Sincroniza el reloj interno con el servidor NTP
12	<code>Firebase.begin()</code>	Inicializa la conexión con Firebase Realtime Database

3.7.2 `loop()` — ejecución continua y concurrente

El `loop()` se ejecuta de forma continua e indefinida. En cada iteración, el ESP32 revisa qué tareas deben ejecutarse según el tiempo transcurrido desde la última vez que cada una corrió, usando el patrón de temporización no bloqueante con `millis()`:

Tabla 26 Estructura del programa: `loop()`

Tarea	Frecuencia	Descripción
Verificar flag <code>wifiNuevo</code>	Cada ciclo	Si BLE recibió nuevas credenciales, aplica reconexión WiFi y reinicia Firebase
Calcular velocidad del viento	Cada 5,000 ms	Lee y reinicia <code>contViento</code> , aplica la fórmula de conversión a km/h
Leer lluvia acumulada	Cada ciclo	Lee y reinicia <code>contLluvia</code> , acumula en <code>lluviaAcumulada</code> en mm
Leer AHT20	Cada ciclo	Obtiene temperatura y humedad mediante <code>getEvent()</code>
Leer BMP280	Cada ciclo	Lee presión cruda, valida, convierte a hPa o activa <code>resetBusI2C()</code>
Leer AS5600	Cada ciclo	Lee ángulo crudo, convierte a grados y punto cardinal
Generar timestamp	Cada ciclo	Obtiene hora actual del reloj NTP y formatea como cadena ISO 8601

Actualizar Serial Monitor	Cada 2,000 ms	Imprime tabla formateada con todos los valores actuales
Actualizar pantalla OLED	Cada 4,000 ms	Cambia a la siguiente pantalla rotativa y renderiza los datos
Enviar a /estacion/actual	Cada 1,500 ms	Serializa datos en JSON y sobrescribe el nodo de tiempo real en Firebase
Guardar en /estacion/historial/	Cada 600,000 ms	Crea nuevo documento con timestamp como clave y reinicia lluviaAcumulada

Al final de cada iteración del loop, el firmware ejecuta `delay(500)` — una pausa de 500 ms que cede tiempo al sistema operativo del ESP32 para gestionar las tareas internas del stack WiFi, BLE y el watchdog del chip.

3.8 Generación de timestamps ISO 8601

Cada registro que se envía a Firebase incluye un timestamp en formato ISO 8601 (YYYY-MM-DDTHH:MM:SS) generado a partir del reloj interno del ESP32, sincronizado con el servidor NTP al arranque. La zona horaria configurada es UTC-6, correspondiente a la Zona Centro de México (Chilpancingo, Guerrero).

La función `getTimestamp()` obtiene el tiempo Unix actual con `time(nullptr)`, lo convierte a hora local con `localtime()` y lo formatea con `strftime()` en la cadena estándar. Este formato permite a Firebase ordenar los registros históricos cronológicamente de forma automática, ya que las cadenas ISO 8601 son lexicográficamente ordenables.

4. CUARTA CAPA: APLICACIÓN

4.1 ¿Qué hace esta capa?

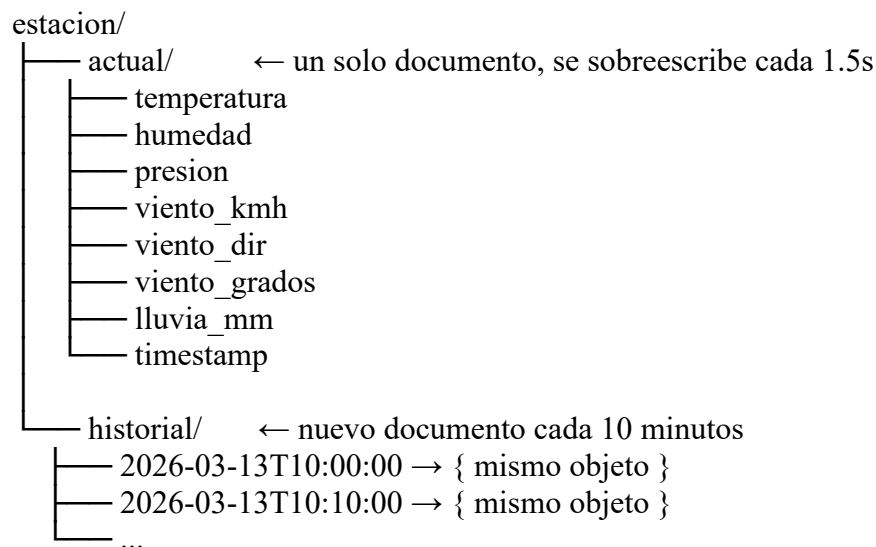
La capa de aplicación es la más cercana al usuario final. Es donde los datos que recorrieron todo el camino — desde los sensores físicos, pasando por el microcontrolador y la red — se convierten en información útil, visualmente comprensible y accionable. Su función no es solo mostrar números, sino presentarlos de forma que cualquier persona pueda entender el estado del clima en tiempo real y analizar su comportamiento histórico. La capa de aplicación representa el nivel más visible del sistema IoT y su diseño determina directamente la utilidad percibida por el usuario final, siendo la interfaz entre los datos procesados y las decisiones humanas que se toman a partir de ellos (Atzori et al., 2010).

En la estación meteorológica esta capa está compuesta por tres componentes: Firebase Realtime Database como backend de datos, una aplicación web accesible desde cualquier navegador, y una aplicación móvil desarrollada en Flutter para Android.

4.2 Backend — Firebase Realtime Database

Firebase actúa como el intermediario entre el ESP32 y las aplicaciones de usuario. Es una base de datos NoSQL en tiempo real desarrollada por Google que almacena los datos en formato JSON y los sincroniza automáticamente con todos los clientes conectados en milisegundos, sin que las aplicaciones tengan que hacer consultas periódicas (Google, 2023).

La estructura de datos está organizada en dos nodos principales:



Esta separación entre datos actuales e históricos es una decisión de diseño clave: permite que el dashboard en tiempo real sea instantáneo (siempre lee un solo documento) mientras que las gráficas históricas consultan los registros acumulados sin saturar la base de datos. A una frecuencia de un registro cada 10 minutos, se generan aproximadamente 144 registros por día y 52,560 por año, una cantidad eficiente y manejable dentro del plan gratuito de Firebase.

4.3. Aplicación web

La aplicación web es un dashboard meteorológico accesible desde cualquier navegador, desarrollada en HTML5, CSS3 y JavaScript ES6 puro sin frameworks adicionales, lo que garantiza carga rápida y compatibilidad universal. Está desplegada en Vercel y disponible también en GitHub Pages.

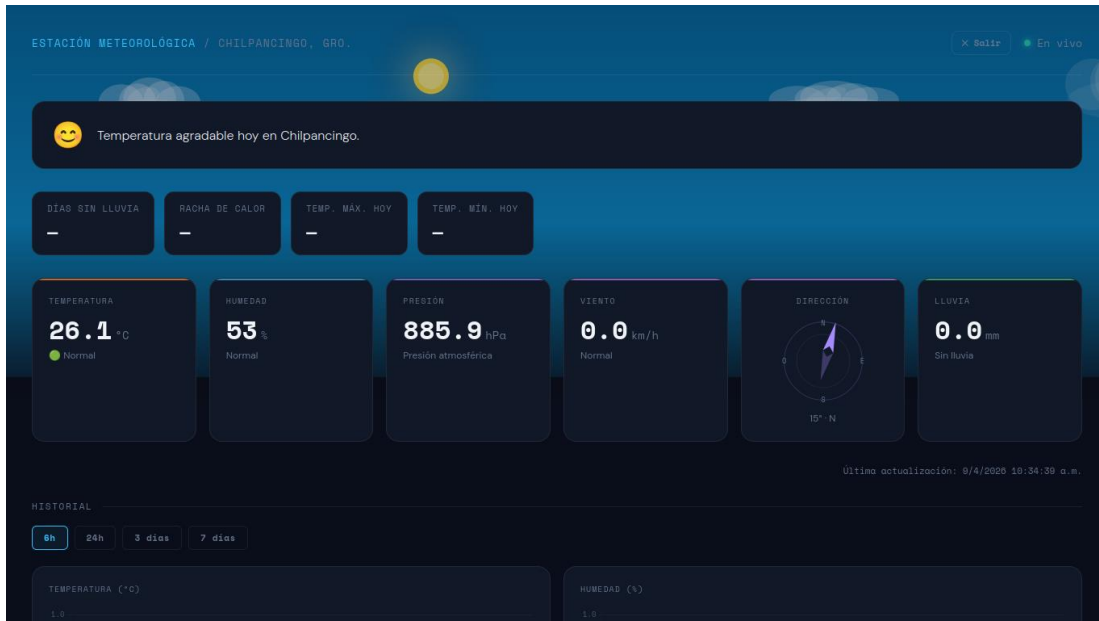


Ilustración 12 Dashboard meteorológico

4.3.1 Inicialización y conexión con Firebase

El primer bloque de código de la aplicación web inicializa Firebase con la configuración del proyecto y obtiene la referencia a la base de datos. A partir de ese momento, la función `onValue()` establece una escucha permanente sobre el nodo `/estacion/actual`: cada vez que el ESP32 escribe

```
import { initializeApp } from
  "https://www.gstatic.com/firebasejs/10.12.0/firebase-app.js";
import { getDatabase, ref, onValue, query, orderByKey, limitToLast }
  from "https://www.gstatic.com/firebasejs/10.12.0/firebase-database.js";

const app = initializeApp(firebaseConfig);
const db = getDatabase(app);

const refActual = ref(db, 'estacion/actual');
onValue(refActual, snap => {
  const d = snap.val();
  if (!d) return;
  // actualizar tarjetas, rosa, frase, alertas...
});
```

Ilustración 13 Inicialización de Firebase

un nuevo dato, Firebase notifica automáticamente al navegador y ejecuta la función de actualización sin necesidad de recargar la página ni hacer peticiones periódicas.

La función `onValue` recibe un snapshot de Firebase cada vez que el nodo cambia. El método `snap.val()` deserializa el JSON recibido en un objeto JavaScript con todos los campos del sensor listos para usarse.

4.3.2 Tarjetas de sensores y transición numérica animada

Cada variable meteorológica se muestra en una tarjeta individual del grid. Para evitar que los números "salten" bruscamente cada 1.5 segundos al recibir nuevos datos, la aplicación implementa una transición animada mediante la función `setValorAnimado()`. Esta función compara el valor anterior con el nuevo y, si cambió, aplica una animación CSS de aparición gradual (flash) antes de actualizar el contenido:

```
const prevVals = {};  
function setValorAnimado(id, nuevoTexto, nuevoNum) {  
  const el = document.getElementById(id);  
  if (prevVals[id] !== undefined && prevVals[id] !== nuevoNum) {  
    el.classList.remove('actualizando');  
    void el.offsetWidth; // fuerza reflow para reiniciar la animación  
    el.classList.add('actualizando');  
  }  
  prevVals[id] = nuevoNum;  
  el.innerHTML = nuevoTexto;  
}
```

Ilustración 14 Tarjetas individuales para mostrar datos

La línea `void el.offsetWidth` fuerza al navegador a recalcular el layout antes de añadir la clase, lo que garantiza que la animación CSS se reinicie correctamente aunque el elemento ya tuviera esa clase aplicada del ciclo anterior.

4.3.3 Fondo dinámico animado con Canvas

La aplicación dibuja el cielo de fondo usando la API Canvas de HTML5, con un bucle de animación continuo activado mediante `requestAnimationFrame`. El estado del cielo se determina automáticamente a partir de los datos del sensor y la hora del sistema, seleccionando entre cinco estados posibles:

Tabla 27 Estados para fondo diáamico con Canvas

Estado (skyState)	Condición de activación	Elementos dibujados
day	Sin lluvia, hora entre 6:00 y 20:00	Gradiente azul, sol animado, nubes en movimiento

night	Sin lluvia, hora entre 20:00 y 6:00	Gradiente oscuro, estrellas titilantes
rain	Lluvia detectada (lluvia_mm > 0)	Gradiente gris, nubes, partículas de lluvia diagonal
storm	Lluvia y viento fuerte simultáneos	Gradiente gris oscuro, lluvia intensa (280 partículas)
Amanecer / atardecer	Hora entre 6–10 o 18–21	Gradiente de colores cálidos (naranja, violeta)

La función `setSkyState()` compara el estado nuevo con el anterior y solo reinicia las partículas cuando el estado cambia, evitando recrear innecesariamente los arreglos de animación:

```
function setSkyState(lluvia, viento, temp) {
  const hr = new Date().getHours();
  let newState;
  if (lluvia > 0 && viento > 30) newState = 'storm';
  else if (lluvia > 0)           newState = 'rain';
  else if (hr < 6 || hr >= 20)   newState = 'night';
  else                           newState = 'day';

  if (newState !== skyState) {
    skyState = newState;
    particles = [];
    if (skyState === 'rain' || skyState === 'storm')
      initRain(skyState === 'storm' ? 280 : 180);
    if (skyState === 'night') initStars();
  }
}
```

Ilustración 15 Función setSkyState

4.3.4 Rosa de los vientos SVG animada

La dirección del viento se visualiza mediante una rosa de los vientos construida en SVG. La flecha es un grupo `<g>` con el atributo `transform-origin` centrado en el punto medio del SVG (50, 50), lo que permite rotarla alrededor de su propio centro. Para evitar giros bruscos al cruzar el norte (transición de 359° a 1°, que sin corrección produciría una rotación de 358° en lugar de 2°), la función `actualizarRosa()` calcula siempre el camino angular más corto mediante interpolación de diferencia:

```

let anguloActual = 0;
function actualizarRosa(grados) {
  let diff = grados - anguloActual;
  if (diff > 180) diff -= 360;
  if (diff < -180) diff += 360;
  anguloActual += diff;
  const flecha = document.getElementById('rosa-flecha');
  flecha.style.transform = `rotate(${anguloActual}deg)`;
}

```

Ilustración 16 Rosa de los vientos animada

La transición CSS `transition: transform 0.8s cubic-bezier(.34,1.56,.64,1)` aplicada al elemento SVG produce un movimiento con rebote suave que comunica visualmente el cambio de dirección de forma natural.

4.3.5 Sistema de alertas y frases contextuales

La aplicación evalúa cinco umbrales sobre los datos recibidos y genera dos tipos de salida: un banner de alerta visible en la parte superior de la pantalla, y una frase en lenguaje natural con un ícono contextual. Los umbrales definidos son:

Tabla 28 Sistema de alertas y frases contextuales

Variable	Umbral	Mensaje generado
Temperatura	< 15 °C	 Está haciendo frío. Abriégate antes de salir.
Temperatura	≥ 30 °C	 Hace calor. Lleva agua contigo.
Temperatura	≥ 35 °C	 Calor extremo. Hidrátate y evita el sol directo.
Humedad	> 80 %	 Alta humedad, puede sentirse más caluroso.
Viento	> 30 km/h	 Viento fuerte. Ten cuidado en moto o bicicleta.
Lluvia	> 5 mm	 Lluvia intensa (alerta en banner)
Lluvia	> 0 mm	 Está lloviendo. ¡No olvides tu paraguas!

4.3.6 Historial — gráficas y tabla de registros

Los datos históricos se consultan con una query de Firebase que ordena por clave (los timestamps ISO 8601 son lexicográficamente ordenables) y limita los resultados según el rango temporal seleccionado. Los registros se filtran del lado del cliente para mostrar únicamente los que caen dentro del rango elegido (6h, 24h, 3 días o 7 días):

```
function cargarHistorial() {
  const limite = Math.ceil(horasRango * 6) + 10;
  const q = query(
    ref(db, 'estacion/historial'),
    orderByKey(),
    limitToLast(limite)
  );
  onValue(q, snap => {
    const corte = Date.now() - horasRango * 3600 * 1000;
    const registros = Object.values(snap.val())
      .filter(r => new Date(r.timestamp).getTime() >= corte)
      .sort((a, b) => new Date(a.timestamp) - new
Date(b.timestamp));
    actualizarGraficas(registros);
    actualizarTabla(registros);
    calcularRachas(registros);
  }, { onlyOnce: true });
}
```

Ilustración 17 Función cargarHistorial() para los datos históricos

Las cuatro gráficas (temperatura, humedad, presión y viento) se construyen con Chart.js sobre elementos <canvas>. La tabla de registros muestra los últimos 50 con un badge de color por temperatura: azul para frío, verde para normal, naranja para calor y rojo para calor extremo.

4.4 Aplicación móvil — Flutter

La aplicación móvil está desarrollada en Flutter con el lenguaje Dart y compilada para Android. Comparte la misma base de datos Firebase que la aplicación web, garantizando que ambas muestren siempre los mismos datos en tiempo real.

4.4.1 Arquitectura de la aplicación

El proyecto sigue una arquitectura de capas con separación de responsabilidades. Cada carpeta tiene una función específica y no se mezclan responsabilidades entre capas:

Tabla 29 Arquitectura de carpetas de la aplicación móvil

Archivo / Carpeta	Responsabilidad
models/weather_data.dart	Define la estructura de datos y la deserialización desde Firebase
providers/weather_provider.dart	Gestiona el estado global, alertas, frases y carga del historial
services/firebase_service.dart	Encapsula la comunicación directa con Firebase
services/notification_service.dart	Gestiona notificaciones push con Firebase Cloud Messaging
screens/home_screen.dart	Pantalla principal con grid de tarjetas en tiempo real
screens/charts_screen.dart	Pantalla de gráficas históricas con fl_chart
screens/history_screen.dart	Listado de registros recientes
widgets/sensor_card.dart	Tarjeta reutilizable para cada variable meteorológica
widgets/frase_card.dart	Widget de frase contextual con ícono
widgets/alert_banner.dart	Banner de alertas activas
theme/app_theme.dart	Colores, tipografía y modo oscuro/claro

4.4.2 Modelo de datos — WeatherData

El modelo WeatherData define la estructura de un registro meteorológico y cómo se construye a partir del mapa JSON que entrega Firebase. El constructor de fábrica fromMap() recibe el Map<dynamic, dynamic> del snapshot de Firebase y extrae cada campo con un valor predeterminado en caso de que alguno llegue nulo, evitando excepciones en tiempo de ejecución:

```

class WeatherData {
    final double temperatura;
    final double humedad;
    final double presion;
    final double vientoKmh;
    final String vientoDir;
    final double vientoGrados;
    final double lluviaMm;
    final DateTime timestamp;

    factory WeatherData.fromMap(Map<dynamic, dynamic> map) {
        return WeatherData(
            temperatura: (map['temperatura'] ?? 0).toDouble(),
            humedad:      (map['humedad']      ?? 0).toDouble(),
            presion:      (map['presion']      ?? 0).toDouble(),
            vientoKmh:    (map['viento_kmh']   ?? 0).toDouble(),
            vientoDir:    map['viento_dir']    ?? '--',
            vientoGrados: (map['viento_grados'] ?? 0).toDouble(),
            lluviaMm:     (map['lluvia_mm']    ?? 0).toDouble(),
            timestamp:    DateTime.tryParse(map['timestamp'] ?? '')
                        ?? DateTime.now(),
        );
    }
}

```

Ilustración 18 Modelo de datos WeatherData

Los nombres de los campos del mapa (temperatura, viento_kmh, etc.) corresponden exactamente con las claves que el ESP32 escribe en Firebase, formando el contrato de datos entre el firmware y la aplicación.

4.4.3 Pantalla principal — HomeScreen

La HomeScreen es un StatelessWidget que obtiene el estado actual desde el WeatherProvider usando `context.watch<WeatherProvider>()`. Cada vez que el provider notifica un cambio — porque Firebase envió un nuevo dato — Flutter reconstruye automáticamente el widget con los valores actualizados.

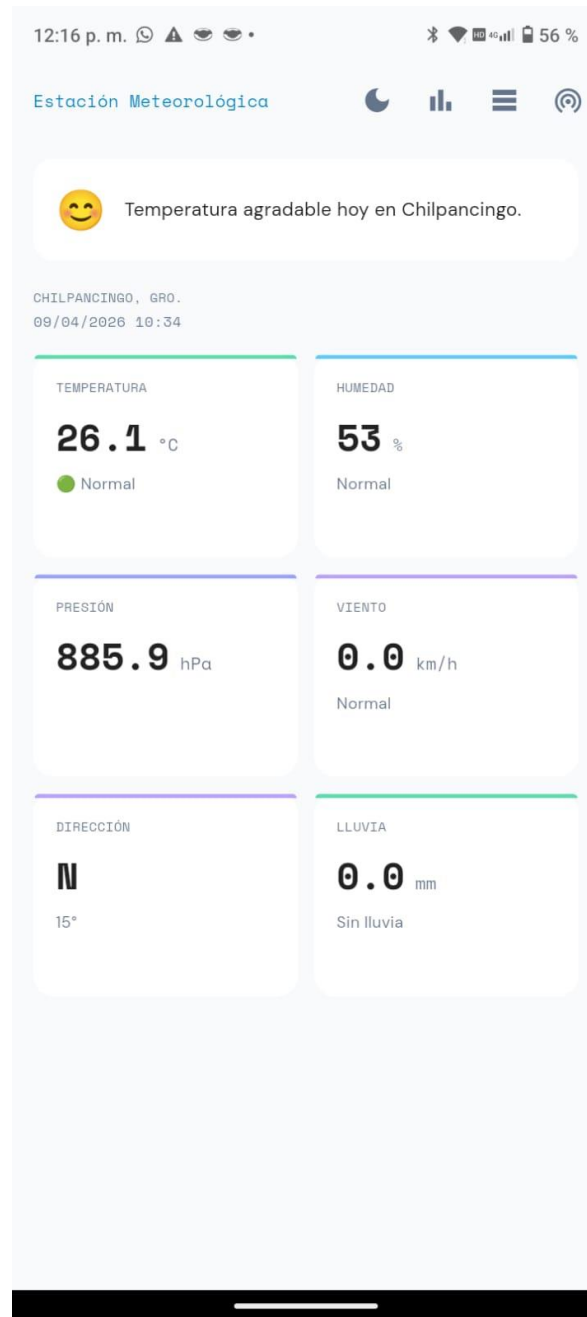


Ilustración 19 HomeScreen de la app Móvil

La pantalla se organiza en una columna vertical con los siguientes elementos, en este orden:

```
Column(children: [
  AlertBanner(alertas: wp.alertas), // banner de alertas activas
  FraseCard(icono: ..., texto: ...), // frase contextual con ícono
  Text('CHILPANCINGO, GRO.'), // ubicación
  Text(fmt.format(d.timestamp)), // timestamp del último dato
  GridView.count( // grid 2x3 de tarjetas
    crossAxisCount: 2,
    children: [
      SensorCard(label: 'Temperatura', ...),
      SensorCard(label: 'Humedad', ...),
      SensorCard(label: 'Presión', ...),
      SensorCard(label: 'Viento', ...),
      SensorCard(label: 'Dirección', ...),
      SensorCard(label: 'Lluvia', ...),
    ],
  ),
])
```

Ilustración 20 StatelessWidget para el HomeScreen

La barra superior (AppBar) incluye tres acciones: el botón de modo oscuro/claro, el acceso a ChartsScreen y el acceso a HistoryScreen. El gesto de pull-to-refresh recarga manualmente el historial llamando a wp.cargarHistorial().

4.4.4 Widget reutilizable — SensorCard

El widget SensorCard es el componente visual central de la aplicación. Es un StatelessWidget completamente reutilizable que recibe como parámetros el nombre de la variable, el valor formateado como texto, la unidad, un subtítulo opcional y un color de acento. Se instancia seis veces en el grid de la HomeScreen, una vez por cada variable meteorológica.

```
class SensorCard extends StatelessWidget {
  final String label;
  final String valor;
  final String unidad;
  final String? subtítulo;
  final Color accentColor;
  ...
}
```

Ilustración 21 Widget reutilizable — SensorCard

Visualmente, cada tarjeta se construye con un Container de esquinas redondeadas. En la parte superior hay una línea de 3 píxeles de altura con el color de acento de la variable — naranja para temperatura, azul para humedad, violeta para presión, etc. Debajo se muestra el nombre de la variable en tipografía Space Mono en mayúsculas, el valor numérico en tamaño grande (28 px en negrita), la unidad en tamaño pequeño junto al número, y opcionalmente el subtítulo contextual en la parte inferior:

```
Container(
  decoration: BoxDecoration(
    color: isDark ? Color(0xFF11827) : Colors.white,
    borderRadius: BorderRadius.circular(16),
  ),
  child: Column(children: [
    // Línea de acento superior
    Container(height: 3, color: accentColor.withOpacity(0.8)),
    Padding(
      padding: EdgeInsets.fromLTRB(16, 14, 16, 16),
      child: Column(children: [
        Text(label.toUpperCase()), // TEMPERATURA
        RichText(text: TextSpan(children: [
          TextSpan(text: valor, style: /* 28px negrita */),
          TextSpan(text: unidad, style: /* 13px muted */),
        ])),
        if (subtitulo != null) Text(subtitulo!),
      ]),
    ),
  ]),
)
```

Ilustración 22 Container para cada tarjeta

El color de fondo de cada tarjeta se adapta automáticamente al modo oscuro o claro del sistema mediante `Theme.of(context).brightness`, sin necesidad de pasar el tema como parámetro.

4.4.5 Sistema de alertas y frases contextuales en Flutter

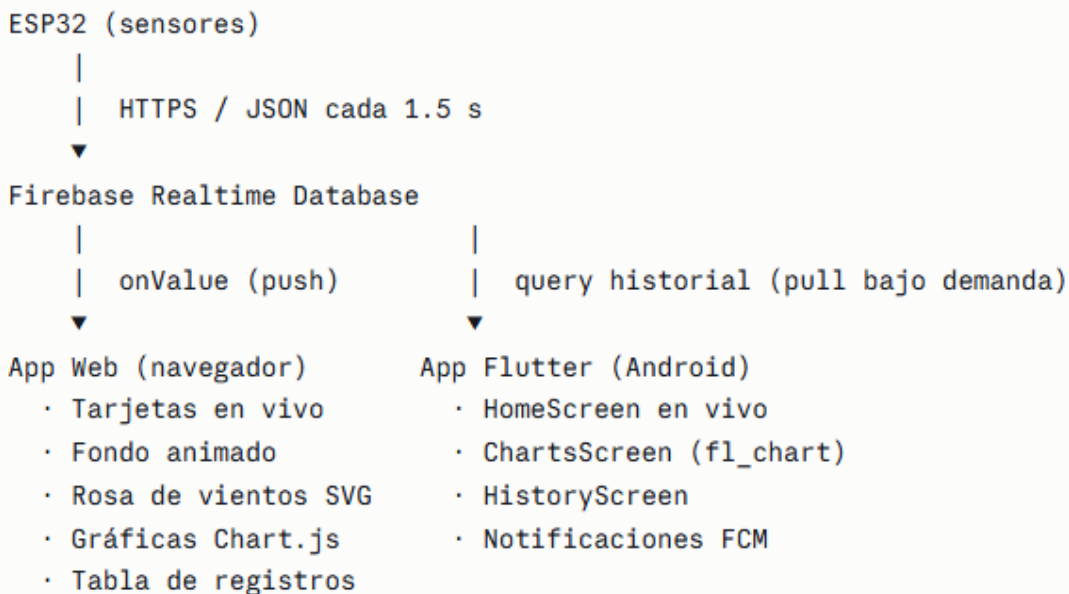
Al igual que en la app web, el `WeatherProvider` evalúa los mismos cinco umbrales sobre los datos recibidos de Firebase y genera dos listas: alertas (lista de cadenas de texto que se pasan al `AlertBanner`) y frase (mapa con las claves icono y texto que se pasan al `FraseCard`). Ambos widgets se reconstruyen automáticamente con cada actualización de Firebase gracias al mecanismo de `Provider`.

4.4.6 Navegación entre pantallas

La app implementa navegación mediante `Navigator.push()` con `MaterialPageRoute`, permitiendo acceder a `ChartsScreen` (gráficas históricas con `fl_chart` y selector de rango 6h / 24h / 3 días / 7 días) e `HistoryScreen` (lista de registros recientes con chips de color por temperatura). La navegación es apilada — el usuario regresa a la `HomeScreen` con el botón de retroceso del sistema.

4.5 Flujo completo de datos en la capa de aplicación

El recorrido completo de un dato desde el sensor hasta la pantalla del usuario sigue este camino:



Tanto la aplicación web como la móvil usan el mecanismo `onValue` de Firebase para el nodo `/estacion/actual`, que establece una conexión `WebSocket` persistente y empuja los datos hacia el cliente cada vez que el ESP32 escribe, sin que las aplicaciones tengan que hacer consultas periódicas. El historial se carga bajo demanda cuando el usuario navega a las gráficas o cambia el rango temporal.

4.6 Despliegue

Tabla 30 Despliegue en distintas plataformas

Plataforma	Tecnología	Acceso
App web	Vercel + GitHub Pages	URL pública desde cualquier navegador
App móvil	APK Android compilado con Flutter	Instalación directa desde archivo APK
Base de datos	Firebase Realtime Database — plan Spark gratuito	Google Cloud (gestionado)

5. CODIGO FUENTE – Repositorio del proyecto en GitHub

El código fuente completo de ambas aplicaciones y del ESP32 está disponible en el repositorio del proyecto:

 <https://github.com/WeatherStationIoT>

Referencias

- Al-Fuqaha, A., Guizani, M., Mohammadi, M., Aledhari, M., & Ayyash, M. (2015). Internet of Things: A survey on enabling technologies, protocols, and applications. *IEEE Communications Surveys & Tutorials*, 17(4), 2347–2376. <https://doi.org/10.1109/COMST.2015.2444095>
- Ashton, K. (2009). That 'Internet of Things' thing. *RFID Journal*, 22(7), 97–114. <https://www.rfidjournal.com/that-internet-of-things-thing>
- Atzori, L., Iera, A., & Morabito, G. (2010). The Internet of Things: A survey. *Computer Networks*, 54(15), 2787–2805. <https://doi.org/10.1016/j.comnet.2010.05.010>
- Banzi, M., & Shiloh, M. (2014). *Getting started with Arduino* (3a ed.). Maker Media.
- Bosch Sensortec. (2020). *BMP280 digital pressure sensor: Data sheet* (versión 1.26). <https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bmp280-ds001.pdf>
- Cattell, R. (2011). Scalable SQL and NoSQL data stores. *ACM SIGMOD Record*, 39(4), 12–27. <https://doi.org/10.1145/1978915.1978919>
- Espressif Systems. (2023). *ESP32 series datasheet* (versión 4.4). https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- Google. (2023). *Firebase Cloud Messaging documentation*. Google Developers. <https://firebase.google.com/docs/cloud-messaging>
- Google. (2023). *Firebase Realtime Database documentation*. Google Developers. <https://firebase.google.com/docs/database>
- Google Developers. (2023). *Flutter documentation: FAQ*. <https://docs.flutter.dev/resources/faq>
- Gubbi, J., Buyya, R., Marusic, S., & Palaniswami, M. (2013). Internet of Things (IoT): A vision, architectural elements, and future directions. *Future Generation Computer Systems*, 29(7), 1645–1660. <https://doi.org/10.1016/j.future.2013.01.010>
- IEEE. (2009). *IEEE standard for information technology — Telecommunications and information exchange between systems — Local and metropolitan area networks — Specific requirements. Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) specifications* (IEEE Std 802.11n-2009). <https://doi.org/10.1109/IEEESTD.2009.5307322>
- IEEE. (2015). *IEEE P2413 — Standard for an architectural framework for the Internet of Things (IoT)*. Institute of Electrical and Electronics Engineers. <https://standards.ieee.org/ieee/P2413/5309>
- MDN Web Docs. (2023). *JavaScript reference*. Mozilla Developer Network. <https://developer.mozilla.org/en-US/docs/Web/JavaScript>

- Mendez, D., Perez, A., Labrador, M., & Barbeau, S. (2020). P-Sense: A participatory sensing system for air pollution monitoring and control. *IEEE International Conference on Pervasive Computing and Communications*, 344–347. <https://doi.org/10.1109/PERCOM.2011.5767598>
- NXP Semiconductors. (2021). *I2C-bus specification and user manual* (revisión 7.0). <https://www.nxp.com/docs/en/user-guide/UM10204.pdf>
- Patel, K. K., Patel, S. M., & Scholar, P. G. (2016). Internet of Things — IoT: Definition, characteristics, architecture, enabling technologies, application and future challenges. *International Journal of Engineering Science and Computing*, 6(5), 6122–6131. <https://doi.org/10.4010/2016.1482>
- Peckol, J. K. (2019). *Embedded systems: A contemporary design tool* (2a ed.). Wiley.
- Rescorla, E. (2018). *The Transport Layer Security (TLS) protocol version 1.3* (RFC 8446). Internet Engineering Task Force. <https://doi.org/10.17487/RFC8446>
- Sensirion. (2020). *AHT20 — Datasheet* (versión 1.0). https://files.seeedstudio.com/wiki/Grove-AHT20_I2C_Industrial_Grade_Temperature_and_Humidity_Sensor/AHT20-datasheet-2020-4-16.pdf
- Zaslavsky, A., Perera, C., & Georgakopoulos, D. (2013). Sensing as a service and big data. *Proceedings of the International Conference on Advances in Cloud Computing*, 21–29. <https://arxiv.org/abs/1301.0159>

Anexos

Tabla de costos del proyecto

Componentes electrónicos

#	Componente	Descripción / Características	Cantidad	Precio unitario (MXN)	Total (MXN)
1	ESP32 DevKit V1	SoC dual-core 240 MHz, WiFi 802.11n + BLE 4.2, 34 GPIO, 4 MB flash	1	\$120.00	\$120.00
2	Módulo AHT20 + BMP280	Sensor dual: temperatura $\pm 0.3^{\circ}\text{C}$, humedad $\pm 2\%$ HR, presión 300–1100 hPa, interfaz I2C	1	\$52.00	\$52.00
3	Encoder magnético AS5600	Posición absoluta 12 bits (4096 pasos/vuelta), interfaz I2C, sin contacto mecánico	1	\$66.00	\$66.00
4	Módulo KY-025 (reed switch) $\times 2$	Sensor magnético digital con comparador LM393, salida DO + AO, LED indicador	2	\$60.00	\$120.00
5	Pantalla OLED SSD1306 0.96"	Resolución 128 $\times$ 64 px, monocromático, interfaz I2C, voltaje 3.3–5 V	1	\$98.00	\$98.00
6	Imán de neodimio	Imán cilíndrico Avellanado	3	\$15.00	\$45.00
7	Cables Dupont hembra-hembra	Cables de conexión 20 cm para protoboard y pines GPIO	1 juego	\$25.00	\$25.00
8	Cables Dupont macho-hembra	Cables de extensión para sensores externos	1 juego	\$25.00	\$25.00
9	Protoboard 830 puntos	Tablilla de pruebas sin soldadura, 830 puntos de conexión	1	\$54.00	\$54.00
10	Cable USB micro-B	Alimentación y programación del ESP32	1	\$30.00	\$30.00
				Subtotal electrónica	\$635.00

Materiales de maqueta y estructura

#	Material	Descripción / Uso en el proyecto	Cantidad	Precio unitario (MXN)	Total (MXN)
11	PVC / tubo	Mástil vertical para montaje de anemómetro y veleta	2 m	\$50	\$50
12	Esparrago, tornillos y tuercas M3	Fijación de componentes y piezas 3D	1 lote	\$70.00	\$70.00
13	Pegamento / sellador	Unión de piezas estructurales y sellado ante humedad	1	\$30	\$30
14	Pintura / acabado	Protección de plc contra sol y humedad exterior	1	\$96	\$96
				Subtotal maqueta	\$246

Piezas impresas en 3D

#	Pieza	Descripción	Filamento	Peso estimado (g)	Costo estimado (MXN)
15	Veleta — cuerpo y soporte	Cuerpo aerodinámico de la veleta con soporte para imán de neodimio en el eje	PLA	~74.9 g	\$123.00
16	Anemómetro — cazoletas y brazo	Tres cazoletas semiesféricas con brazos de montaje para el eje del reed switch	PLA	~73 g	\$117.00
17	Base / gabinete del ESP32	Carcasa de protección para la placa ESP32 y conexiones	PLA	~182.63 g	\$206.00
18	Stevenson screen para el sensor de temp		PLA	~70.46 g	\$151.00
19	Embudo del pluviómetro Soporte del sensor de lluvia	Embudo colector de 100 cm ² de área para el balancín del pluviómetro	PLA	~85.83 g	\$156.00
			Subtotal impresión 3D (filamento PLA ~\$450/kg)	~486.82 g	\$753.00

Resumen general de costos

Categoría	Total (MXN)
Componentes electrónicos	\$635.00
Materiales de maqueta y estructura	\$246
Piezas impresas en 3D (filamento PLA)	\$753.00
Servicios y plataformas digitales	\$0.00
TOTAL ESTIMADO DEL PROYECTO	~\$1,634